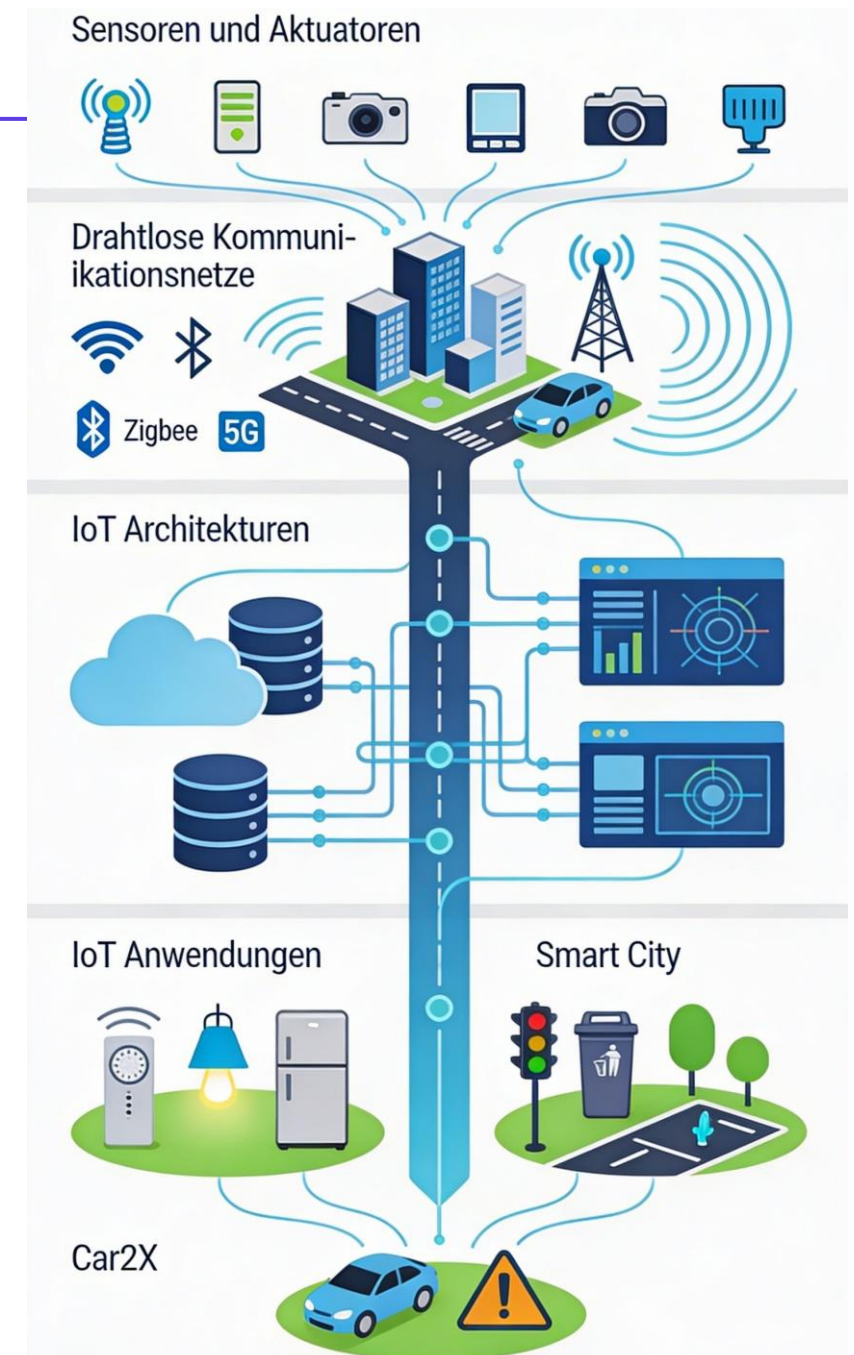




IoT-Schichten und Architekturen

IoT Infrastruktur: Berechnungs- und Architekturmodelle

Skalierbarkeit als zentrales Problem der IoT-Vision

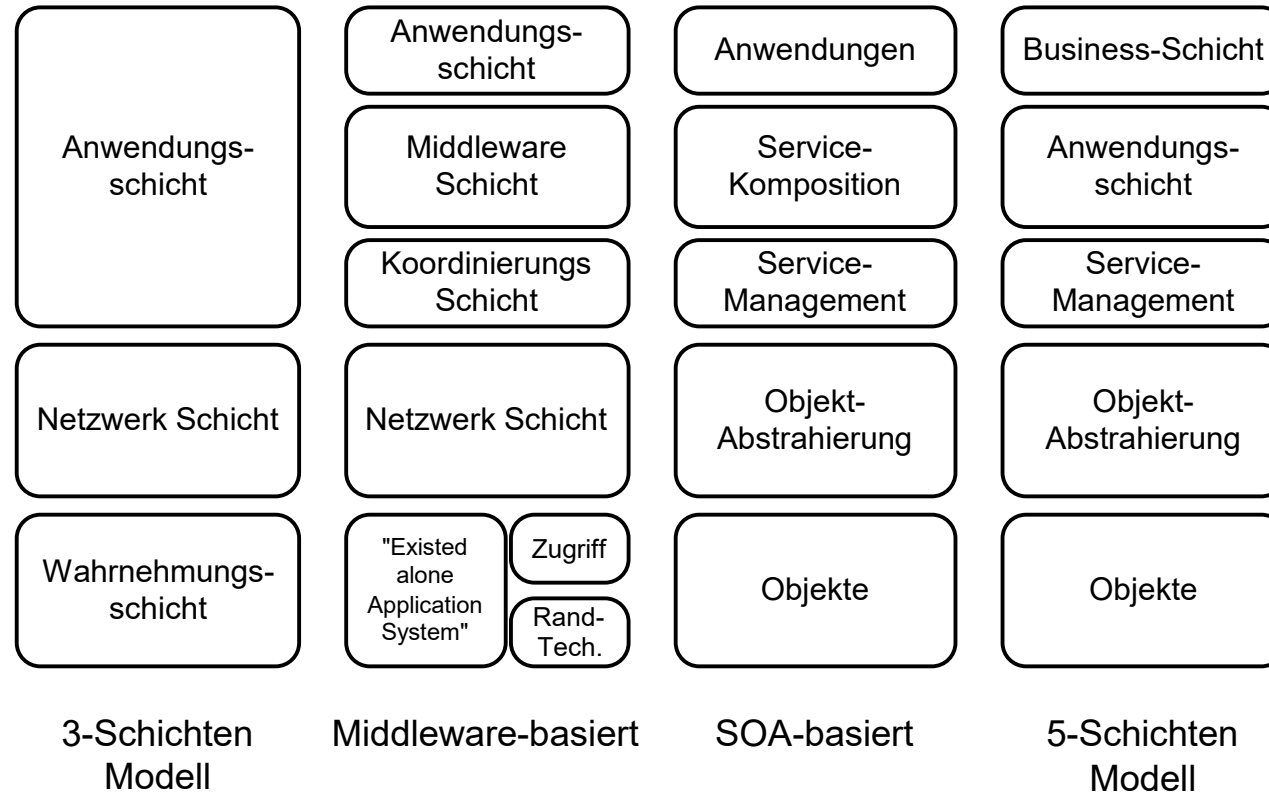
- Milliarden v. Datengeneratoren
- Sensoren besitzen stark begrenzte Ressourcen
- Datenweiterleitung erzeugt Overhead
- Real-Time Anforderungen für bestimmte Szenarien

Ideen für IoT-Architekturen

- Nutzung von Cloud-Diensten
- Lokale Berechnung wenn sinnvoll
- Auslagerung wenn nötig
- Cloud-basierte Architekturen bieten Plattformen, Software und Infrastruktur „as a Service“ an (Everything-as-a-Service)

Die hohe Anzahl und geringe Rechenkraft von Sensoren macht eine Delegation der Datenverarbeitung bei rechenintensiven Services notwendig. Cloud-basierte Architekturen sind dafür geeignet.

Architektur - IoT Schichtenmodelle

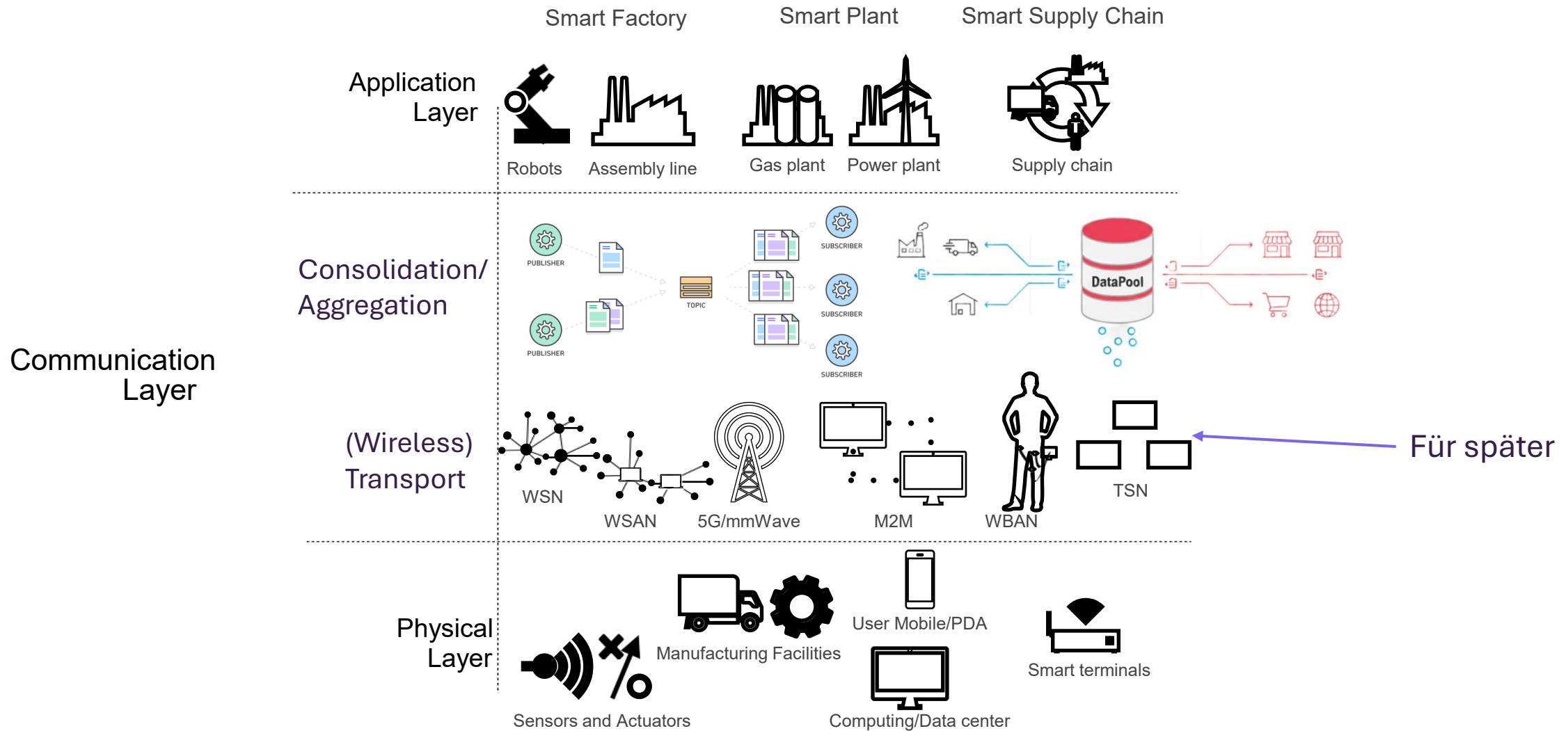


Schichtenbasierte Architekturen für IoT

- Verschiedene mögliche Sichtweisen
- **Objekte:** Sensoren, Aktoren
- **Objekt-Abstrahierung:** Kommunikation d. Objektdaten (z.B. via RFID)
- **Service Management:** Middleware-Schicht, verbindet Dienste und Anfragende
- **Anwendungsschicht:** Bereitstellung der Dienste
- **Business-Schicht:** Visualisierung und Zusammenfassung von Daten

Ähnlich den Netzwerk-Schichtenmodellen existieren auch im Bereich IoT mehrere Modellevarianten.

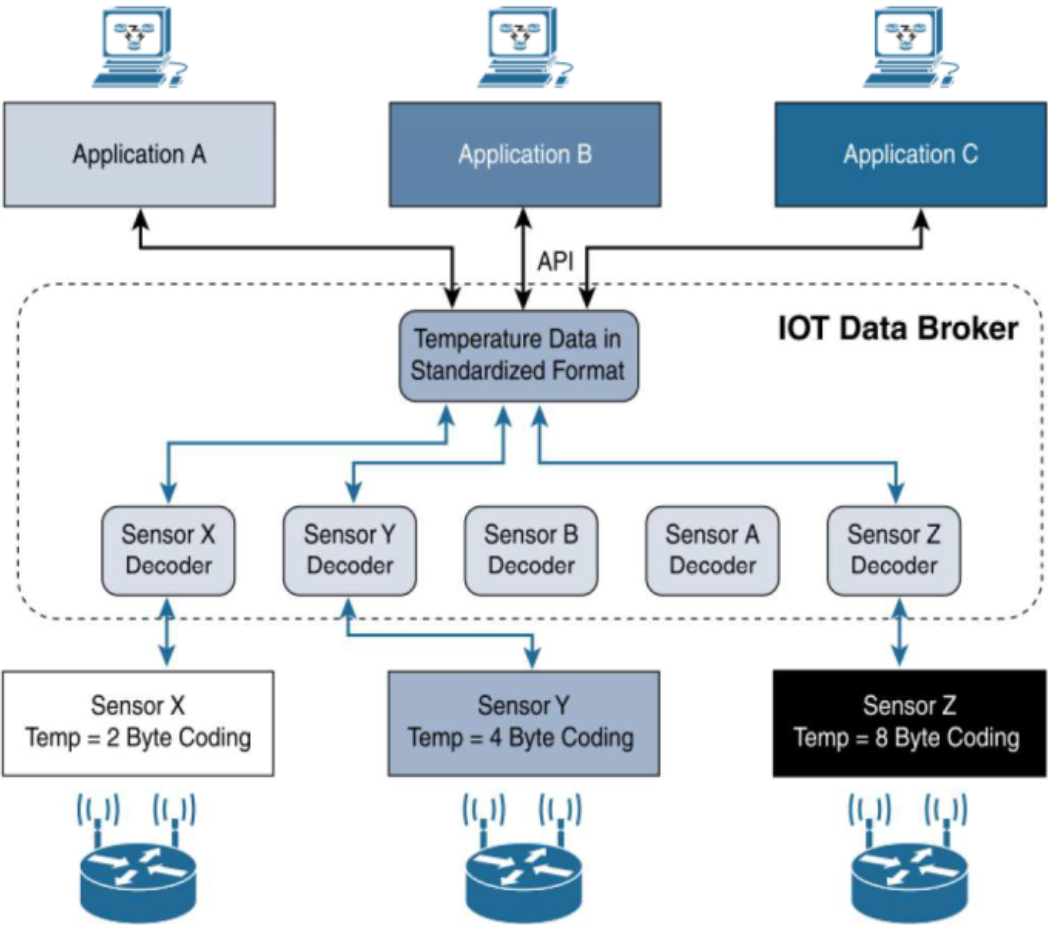
Architektur - IoT Schichtenmodelle



Eine gewissen Tendenz ist zu 3-Schichten-Varianten festzustellen.

Quelle: H. Xu. et al (2018)

IoT Data Broker und Anwendungsschicht

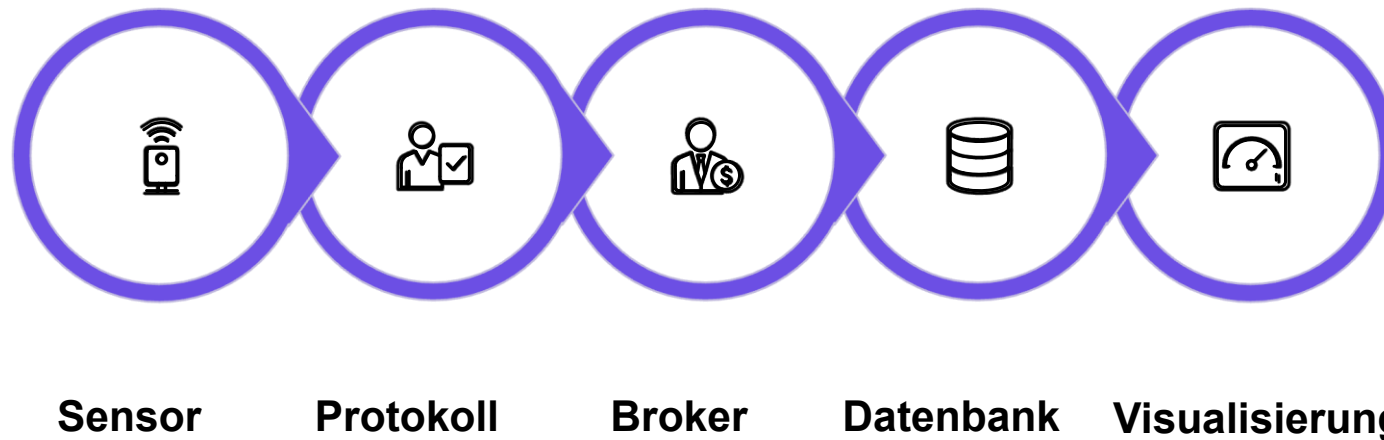


CoAP	MQTT
UDP	TCP
IPv6	
6LoWPAN	
802.15.4 MAC	
802.15.4 PHY	

IoT Data Broker

Von Sensordaten zur Auswertung: Der Weg durch den Stack

In dieser Vorlesung betrachten wir, wie Rohdaten von IoT-Sensoren erfasst, übertragen, gespeichert und schließlich visualisiert werden — von der Entstehung am Gerät bis zur Auswertung im Dashboard.



Dieser Datenfluss zeigt den Weg der Informationen von der physischen Welt der Sensoren bis hin zur digitalen Darstellung in Dashboards, wobei jeder Schritt eine spezifische Funktion im IoT-Ökosystem erfüllt.

i Diese Vorlesung fokussiert auf die Anwendungsschicht (Layer 7 des OSI-Modells): REST, CoAP, MQTT, HTTP. Drahtlose Übertragungsstandards wie WLAN, Zigbee, LoRa oder BLE werden hier nicht behandelt — sie sind Thema einer separater Betrachtung.

Fokus: Anwendungsschicht — nicht die Funkebene

IoT-Kommunikation findet auf mehreren Schichten statt. Diese Vorlesung betrachtet ausschließlich die Protokolle der Anwendungsschicht — also wie Daten strukturiert, adressiert und ausgetauscht werden, nicht wie sie physisch übertragen werden.

Was wir NICHT behandeln (Funk- & Übertragungsebene)

WLAN / Wi-Fi (IEEE 802.11): Physische Übertragung im 2,4/5 GHz Band

Zigbee (IEEE 802.15.4): Mesh-Netz für Heimautomation

LoRa / LoRaWAN: Weitreichende Niedrigenergie-Übertragung

Bluetooth / BLE: Kurzreichweitige Gerätekommunikation

Z-Wave: Proprietäres Mesh-Protokoll für Smart Home

NB-IoT / LTE-M: Mobilfunkbasierte IoT-Übertragung

Was wir behandeln (Anwendungsschicht / OSI Layer 7)

REST / HTTP: Ressourcenorientierte Web-APIs

CoAP: Leichtgewichtiges REST für Microcontroller

MQTT: Publish/Subscribe für IoT-Datenströme

JSON / XML: Nachrichtenformate und Serialisierung

InfluxDB Line Protocol: Zeitreihendaten schreiben

Flux / InfluxQL: Zeitreihendaten abfragen

i Die Anwendungsschicht definiert, WAS kommuniziert wird und WIE Daten strukturiert sind. Die Übertragungsschicht definiert, WIE Bits physisch übertragen werden. Beide Ebenen sind unabhängig voneinander — ein MQTT-Broker läuft über WLAN genauso wie über LoRaWAN.

REST APIs im IoT: Offene Schnittstellen für vernetzte Systeme

Was ist eine REST API?

- REST = Representational State Transfer
- Architekturstil für verteilte Systeme (Roy Fielding, 2000)
- Basiert auf HTTP — dem Protokoll des Web
- Zustandslos: jede Anfrage enthält alle nötigen Informationen
- Ressourcen werden über URLs adressiert
- Antworten in maschinenlesbaren Formaten (JSON, XML)

Warum REST im IoT?

- Geräte wie Sensoren, Aktoren, Gateways bieten HTTP-Endpunkte an
- Keine proprietäre Middleware nötig
- Direkte Integration in Web-Backends, Cloud-Dienste, Dashboards
- Weit verbreitet: jeder HTTP-Client kann kommunizieren
- Einfaches Debugging mit Browser oder curl

 REST APIs sind der de-facto-Standard für offene IoT-Geräte. Systeme wie Tasmota, Shelly oder Home Assistant setzen vollständig auf HTTP-basierte Schnittstellen.

REST API Calls: HTTP-Methoden und Tasmota-Beispiele

HTTP-Methoden im Überblick

01

GET

Ressource lesen / Zustand abfragen

02

POST

Neue Ressource anlegen / Aktion auslösen

03

PUT

Ressource vollständig ersetzen / Konfigurieren

04

DELETE

Ressource entfernen

Tasmota: Praxisbeispiele

```
# Gerätestatus abfragen
GET http://192.168.1.100/cm?cmnd=Status

# Relais einschalten
GET http://192.168.1.100/cm?cmnd=Power%20On

# Sensorwert lesen (Temperatur)
GET http://192.168.1.100/cm?cmnd=Status%2010

# Konfiguration setzen (MQTT-Broker)
GET http://192.168.1.100/cm?cmnd=MqttHost%20192.168.1.50

# Antwort (JSON):
{
  "Status": {
    "Module": 1,
    "FriendlyName": ["Tasmota"],
    "Power": 1,
    "Uptime": "0T12:34:56"
  }
}
```



Tasmota ist eine quelloffene Firmware für ESP8266/ESP32-basierte Geräte. Alle Funktionen sind über eine einfache HTTP-GET-API steuerbar — kein SDK, kein proprietäres Protokoll.

JSON: Das universelle Nachrichtenformat im IoT

JSON — Eigenschaften

- JavaScript Object Notation (JSON)
- Menschenlesbar und maschinenverarbeitbar
- Schlüssel-Wert-Paare, Arrays, verschachtelte Objekte
- Kein Schema erforderlich (flexibel, aber fehleranfällig)
- Nativ in JavaScript, Python, Java, C++ unterstützt
- Kompakter als XML, aber ausführlicher als Binärformate
- MIME-Type: application/json

JSON in der Praxis: IoT-Sensordaten

```
{
  "device": "sensor-42",
  "location": "Serverraum-B",
  "timestamp": "2026-05-19T10:30:00Z",
  "readings": {
    "temperature": 22.4,
    "humidity": 58.1,
    "pressure": 1013.25
  },
  "status": "ok",
  "alerts": [],
  "firmware": "v2.1.3"
}
```



JSON ist das Standardformat für REST APIs, MQTT-Payloads und Cloud-Dienste. Tasmota, Shelly und die meisten modernen IoT-Geräte liefern ihre Antworten als JSON.

XML: Strukturiertes Nachrichtenformat mit Schema-Unterstützung

XML — Eigenschaften

- eXtensible Markup Language (XML)
- Hierarchische Baumstruktur mit Tags und Attributen
- Schema-Validierung möglich: XSD, DTD
- Namespaces für Kollisionsvermeidung in komplexen Systemen
- Ausführlicher als JSON — höherer Overhead
- Weit verbreitet in industriellen Protokollen:
 OPC-UA, SOAP, SensorML
- MIME-Type: application/xml

XML in der Praxis: IoT-Sensordaten

```
<?xml version="1.0" encoding="UTF-8"?>
<SensorData>
  <Device id="sensor-42">
    <Location>Serverraum-B</Location>
    <Timestamp>2026-05-19T10:30:00Z</Timestamp>
    <Readings>
      <Temperature unit="°C">22.4</Temperature>
      <Humidity unit="%">58.1</Humidity>
      <Pressure unit="hPa">1013.25</Pressure>
    </Readings>
    <Status>ok</Status>
    <Firmware>v2.1.3</Firmware>
  </Device>
</SensorData>
```

❗ XML ist in industriellen und behördlichen Systemen (SCADA, OPC-UA, SensorML) nach wie vor dominant. Der höhere Overhead wird durch strikte Validierbarkeit und Interoperabilität gerechtfertigt.

Anwendungsschicht herkömmlich: REST über HTTP

Representational State Transfer (REST):

- Zum Zugriff auf Informationen über das HyperText Transfer Protocol (HTTP)
- Zugriffsoptionen: List, Retrieve, Replace, Update, Create, Delete

Eigenschaften: Client/Server Architektur, zustandslos, cachebar

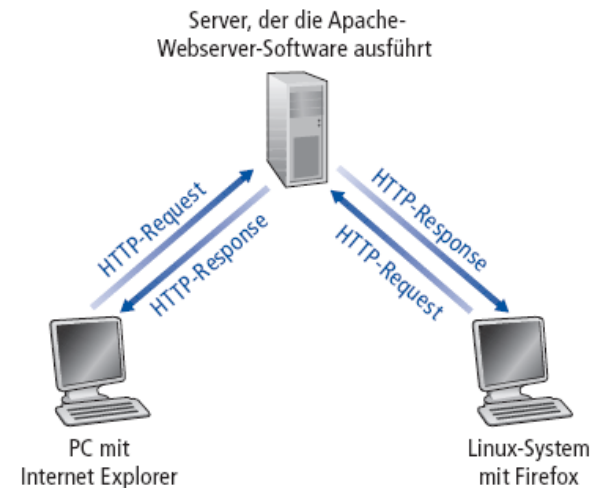
HTTP-Request-Nachricht: ASCII (vom Menschen leicht zu lesen)

Request-Zeile
(GET, POST,
HEAD Befehle)

Header-Zeilen

GET /resources/gps HTTP/1.1
Host: api.citydevice.com
User-agent: Mozilla/4.0
Connection: close
Accept-language: de

Zusätzlicher Wagenrücklauf +
Zeilevorschub zeigt das Ende der Nachricht an



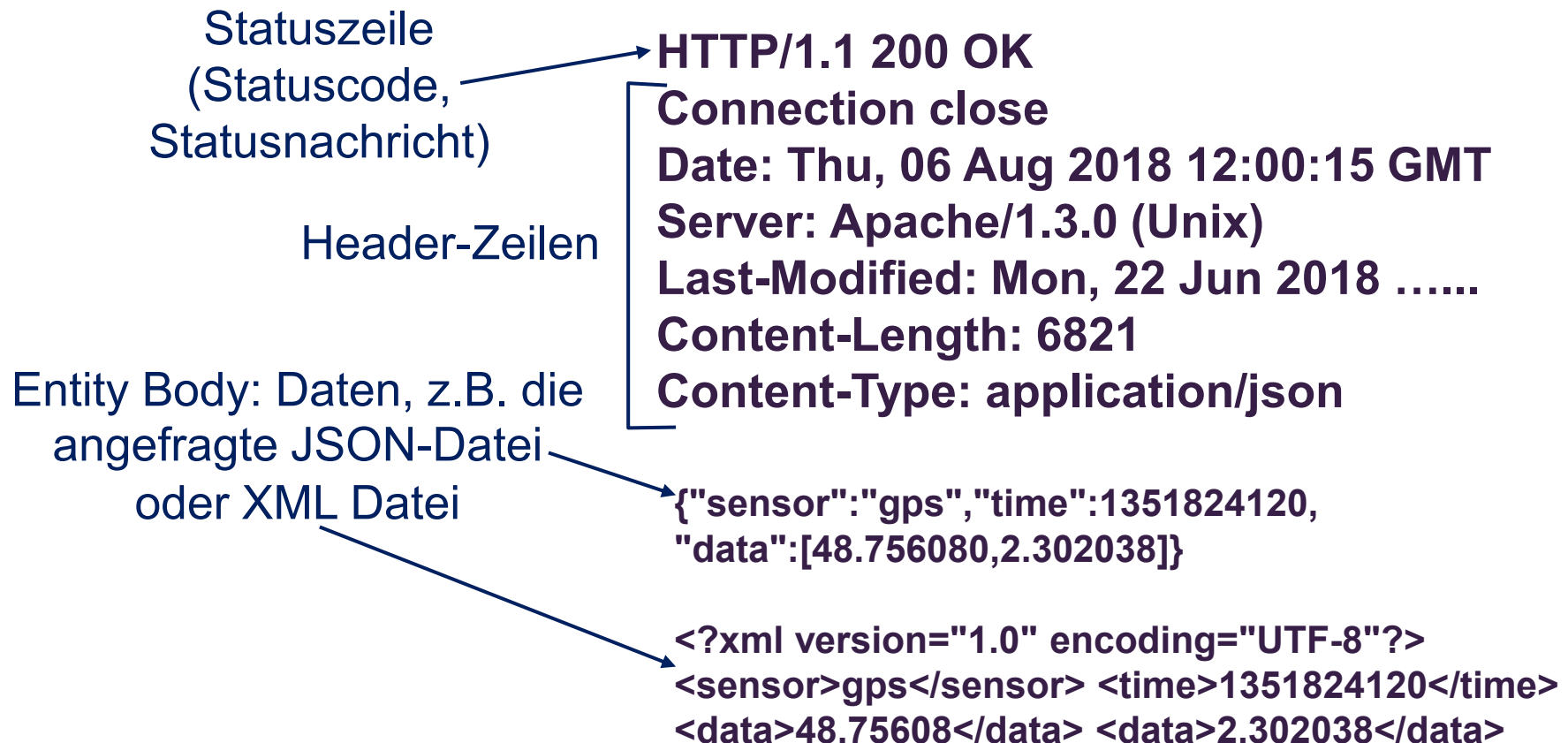
Beispiel: HTTP-Response-Nachricht

Annahme: Lokationsdaten sollen übermittelt werden

Option 1: HTML – Hypertext Markup Language

Option 2: JSON – JavaScript Object Notation

Option 3: XML – eXtensible Markup Language



Offenheit als Systemstrategie: offene Schnittstellen

Offene IoT-Systeme wie Tasmota, Shelly oder ESPHome sind nicht nur günstiger
— sie sind strategisch überlegen, weil sie sich nahtlos in beliebige Systemlandschaften integrieren lassen.

Keine Vendor-Lock-in

Proprietäre Systeme binden Betreiber an einen Hersteller.
Offene REST APIs erlauben den Wechsel von Backend, Cloud oder Dashboard ohne Gerätetausch.

Universelle Anbindung

Jedes System mit HTTP-Client kann offene IoT-Geräte ansprechen: Node-RED, Python-Skripte, Home Assistant, AWS IoT, Azure IoT Hub, eigene Microservices.

Composability

Offene Systeme lassen sich zu komplexen Architekturen komponieren: Sensordaten → REST → Node-RED → MQTT → InfluxDB → Grafana — jede Schicht austauschbar.

Beispiel: Tasmota in einer Unternehmensarchitektur

- Tasmota-Gerät sendet Daten per HTTP an Node-RED
- Node-RED transformiert und leitet an MQTT-Broker weiter
- Parallel: direkter REST-Call aus Python-Skript für Alarmlogik
- Grafana liest aus InfluxDB — kein Tasmota-spezifisches Plugin nötig

Weitere offene IoT-Systeme

Shelly: HTTP + MQTT + CoAP
ESPHome: YAML-konfigurierbar, REST + MQTT
Home Assistant: REST API + Webhooks
OpenHAB: REST API, breite Protokollunterstützung
Zigbee2MQTT: Zigbee-Geräte über MQTT

CoAP: Constrained Application Protocol

CoAP ist ein spezialisiertes Web-Protokoll für ressourcenbeschränkte Geräte (Microcontroller, Sensoren)
— entwickelt von der IETF als leichtgewichtige Alternative zu HTTP für das Internet of Things.

CoAP — Kernmerkmale

- RFC 7252 (IETF, 2014)
- Basiert auf UDP statt TCP → kein Verbindungsaufbau-Overhead
- Binäres Nachrichtenformat → minimale Paketgröße (4 Byte Header)
- REST-ähnliche Semantik: GET, POST, PUT, DELETE
- Asynchrones Request/Response-Modell
- Observe-Mechanismus:
Push-Benachrichtigungen bei Wertänderung
- DTLS für Verschlüsselung (UDP-basiertes TLS)
- Standard-Port: 5683 (CoAP), 5684 (CoAPS)

Zielplattformen und Einsatzszenarien

6LoWPAN / IEEE 802.15.4

Drahtlose Sensornetze mit IPv6

Constrained Devices

Microcontroller mit < 10 KB RAM

Smart Metering

Energiezähler, Wasserzähler

Industrielle Sensorik

Temperatur, Druck, Vibration

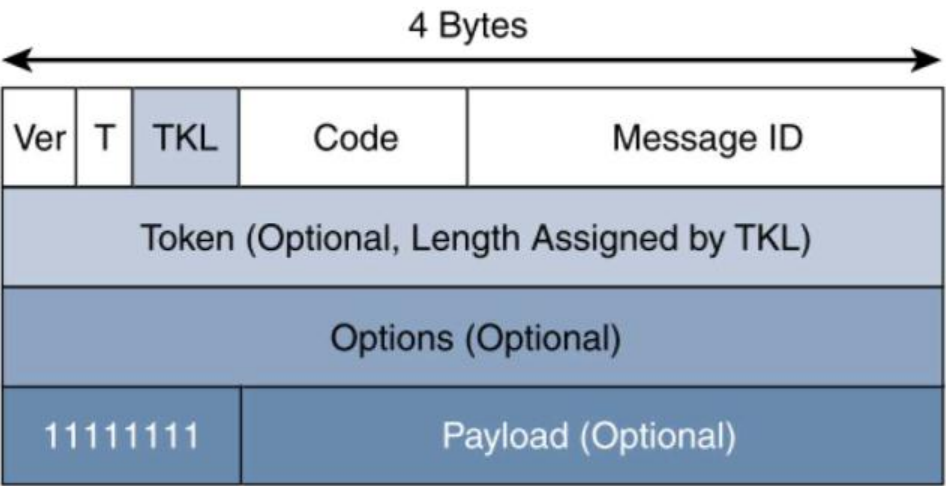
Smart Building

Beleuchtung, HLK-Steuerung

❗ CoAP ist in RFC 7252 standardisiert und wird von der IETF CoRE (Constrained RESTful Environments) Working Group gepflegt. Es ist das Protokoll der Wahl für IPv6-basierte Sensornetze.

CoAP - Nachrichtenformat

CoAP Message Field	Description
Ver (Version)	Identifies the CoAP version.
T (Type)	Defines one of the following four message types: Confirmable (CON), Non-confirmable (NON), Acknowledgement (ACK), or Reset (RST). CON and ACK are highlighted in more detail in Figure 6-9.
TKL (Token Length)	Specifies the size (0–8 Bytes) of the Token field.
Code	Indicates the request method for a request message and a response code for a response message. For example, in Figure 6-9, GET is the request method, and 2.05 is the response code. For a complete list of values for this field, refer to RFC 7252.
Message ID	Detects message duplication and used to match ACK and RST message types to CON and NON message types.
Token	With a length specified by TKL, correlates requests and responses.
Options	Specifies option number, length, and option value. Capabilities provided by the Options field include specifying the target resource of a request and proxy functions.
Payload	Carries the CoAP application data. This field is optional, but when it is present, a single byte of all 1s (0xFF) precedes the payload. The purpose of this byte is to delineate the end of the Options field and the beginning of Payload.



Effizienzvergleich: CoAP vs. HTTP

Merkmal	HTTP/REST	CoAP
Transportprotokoll	TCP	UDP
Minimaler Header	~200 Byte (HTTP-Header)	4 Byte
Verbindungsaufbau	TCP-Handshake (3-Way)	Kein Handshake
Nachrichtengröße (Sensor)	~500 Byte	~20–50 Byte
RAM-Bedarf (Client)	> 50 KB	< 10 KB
Verschlüsselung	TLS	DTLS
Push-Support	Nein (Polling)	Ja (Observe)
Multicast	Nein	Ja (UDP)

Auf einem ESP8266 mit 80 KB RAM ist ein vollständiger HTTP-Stack kaum realisierbar. CoAP läuft problemlos auf Microcontrollern mit 8 KB RAM — ein entscheidender Vorteil für batteriebetriebene Sensorknoten.

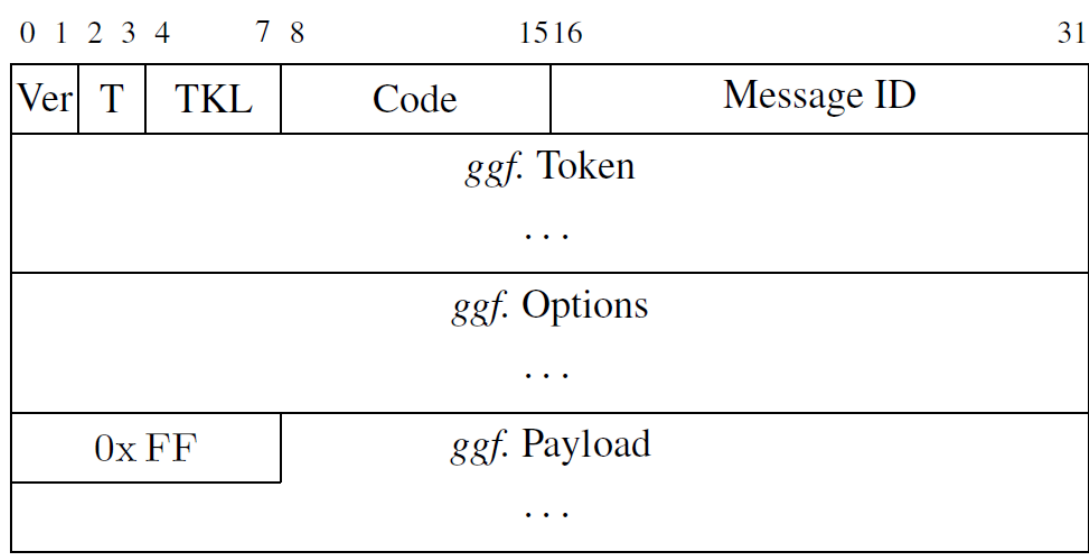
Anwendungsschicht für Smart Cities: CoAP

Constrained Application Protocol – CoAP

- Ziel: Kommunikation von ressourcenbeschränkten Geräten in verlustbehafteten Netzwerken
- Nutzt UDP (nicht zuverlässig, unsicher aber mit kurzem 8-Byte-Header)

Nachrichtenformat

- 4-Byte Header sowie Token, Optionen (Zusatzparameter), Nutzdaten
- T (Nachrichtentyp) und Message ID zum Umsetzen von:
 - Zuverlässigkeits-, Koordinations-, Staukontroll- und Deduplizierungsmechanismen
 - T codiert: Confirmable, Non-Confirmable, Acknowledgement, Reset



Nr.	Name	Länge [B]
1	If-Match	0-8
3	Uri-Host	1-255
4	ETag	1-8
5	If-None-Match	0
7	Uri-Port	0-2
8	Location-Path	0-255
11	Uri-Path	0-255
12	Content-Format	0-2
14	Max-Age	0-4
15	Uri-Query	0-255
16	Accept	0-2
20	Location-Query	0-255
35	Proxy-Uri	1-1034
39	Proxy-Scheme	1-255

CoAP: Anfrage / Antwort Interaktionsmodell

Code-Feld 8-Bit = c.dd (3 Bits für Class, 5 Bits für Details)
im Header codieren alle Nachrichtentypen

Klasseneinteilung:

- Anfragen (0.dd)
- Success (2.dd)
- Client-Error (4.dd)
- Server-Error (5.dd)
- Ausreichend Codes reserviert

Code:	0.01	0.02	0.03	0.04
Name:	GET	POST	PUT	DELETE

CoAP-Codes von Anfragemethoden

Code	Beschreibung	Code	Beschreibung
2.01	Created	4.05	Method Not Allowed
2.02	Deleted	4.06	Not Acceptable
2.03	Valid	4.12	Precondition Failed
2.04	Changed	4.13	Request Entity Too Large
2.05	Content	4.15	Unsupported Content-Format
4.00	Bad Request	5.00	Internal Server Error
4.01	Unauthorized	5.01	Not Implemented
4.02	Bad Option	5.02	Bad Gateway
4.03	Forbidden	5.03	Service Unavailable
4.04	Not Found	5.04	Gateway Timeout
		5.05	Proxying Not Supported

CoAP-Antwortcodes

CoAP bietet auch HTTP-Proxying, Multicasting, Service & Resource Discovery und kann mittels Sicherheitsmechanismen abhörsicher gemacht werden (trotz UDP).

REST vs. CoAP vs. MQTT: Protokollwahl im IoT

Die Wahl des richtigen Protokolls hängt von Gerätekategorie, Netzwerkbedingungen und Anforderungen an Zuverlässigkeit und Echtzeitfähigkeit ab.

Kriterium	REST/HTTP	CoAP	MQTT
Transportschicht	TCP	UDP	TCP
Architektur	Request/Response	Request/Response + Observe	Publish/Subscribe
Header-Overhead	Hoch (~200 B)	Minimal (4 B)	Gering (2 B)
Gerätetyp	Gateway, Server	Microcontroller, Sensor	Alle IoT-Klassen
RAM-Bedarf	> 50 KB	< 10 KB	~5–20 KB
Push-Fähigkeit	Nein (Polling)	Ja (Observe)	Ja (nativ)
Multicast	Nein	Ja	Nein
Verschlüsselung	TLS	DTLS	TLS
Standardisierung	IETF HTTP	RFC 7252	OASIS MQTT 5.0
Typischer Einsatz	Tasmota, Shelly, Cloud APIs	6LoWPAN, Smart Metering	MING-Stack, IIoT

Wähle REST wenn...

Gerät hat ausreichend RAM, Integration in Web-Ökosystem gewünscht, einfaches Debugging wichtig

Wähle CoAP wenn...

Batteriebetrieb, < 10 KB RAM, IPv6-Sensornetz, Multicast-Bedarf

Wähle MQTT wenn...

Viele Geräte, Pub/Sub-Architektur, zuverlässige Nachrichtenübertragung (QoS), MING-Stack

Der MING-Stack

MQTT · Node-RED · InfluxDB · Grafana

Eine praxisorientierte Einführung in die Architektur moderner IoT-Dateninfrastrukturen — von der Datenentstehung am Sensor bis zur Visualisierung im Dashboard. Diese Vorlesung behandelt die technischen Grundlagen, Systemintegration sowie relevante Alternativen zu jeder Komponente des Stacks.



M

i

N

G

Lernziele und Gliederung

Nach dieser Vorlesung können Studierende den MING-Stack als Gesamtarchitektur beschreiben, jede Komponente technisch einordnen und begründete Technologieentscheidungen für reale IoT-Szenarien treffen.

01

Motivation & Überblick

IoT-Datenflüsse, Architekturprinzipien, Systemübersicht

02

MQTT

Protokolldesign, QoS-Levels, Broker-Konzept, Alternativen

03

Node-RED

Flow-basierte Programmierung, Einsatz, Grenzen, Alternativen

04

InfluxDB

Zeitreihendatenbanken, Datenmodell, Abfragesprachen, Alternativen

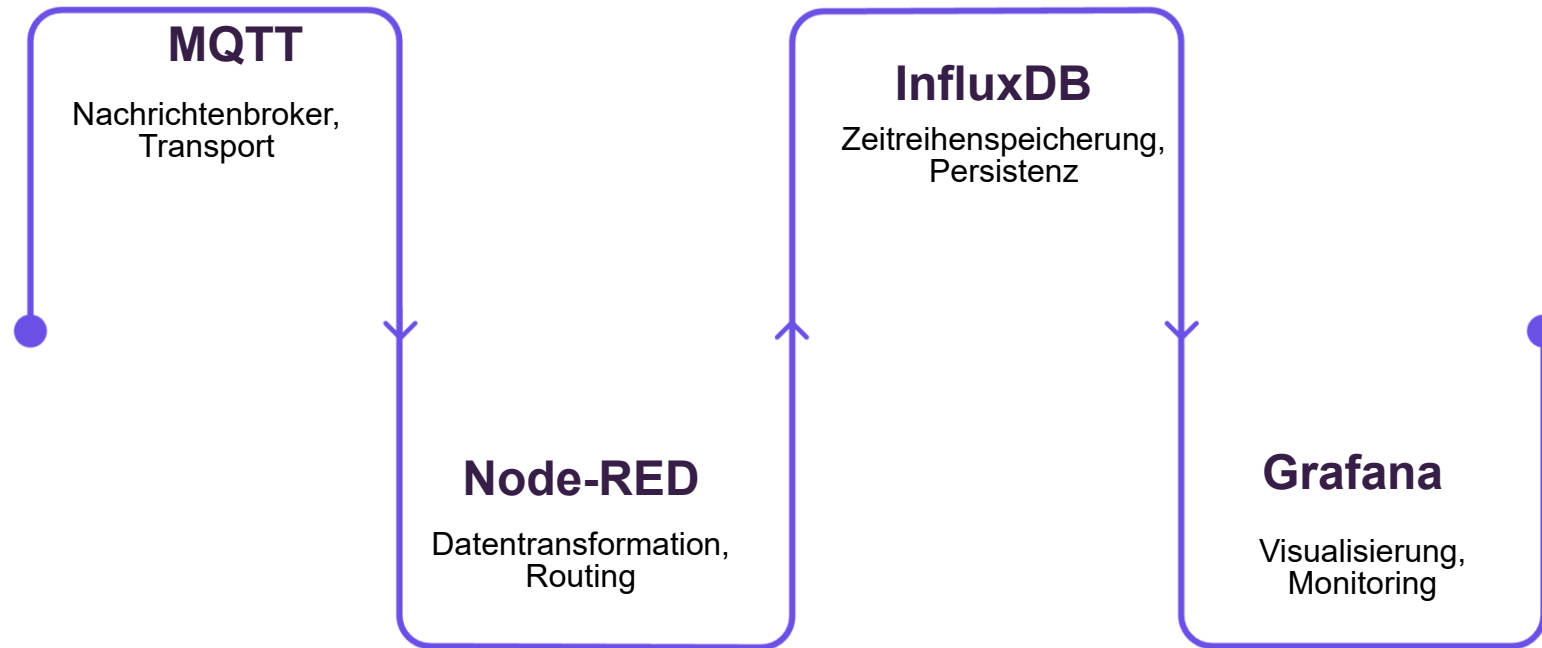
05

Grafana

Visualisierungskonzepte, Datenquellen, Alerting, Alternativen

Der MING-Stack: Systemarchitektur im Überblick

Der MING-Stack ist eine etablierte Open-Source-Referenzarchitektur für IoT-Dateninfrastrukturen. Jede Komponente übernimmt eine klar abgegrenzte Aufgabe im Datenpipeline-Prozess.



Die Stärke des Stacks liegt in der losen Kopplung der Komponenten: Jede Schicht ist austauschbar, ohne das Gesamtsystem zu destabilisieren. Dieses Prinzip der **Separation of Concerns** ermöglicht flexible, wartbare IoT-Architekturen.



MQTT: Message Queuing Telemetry Transport

Protokollkernmerkmale

- Publish/Subscribe-Paradigma
- Leichtgewichtig: minimaler Header (2 Byte)
- TCP/IP-basiert (Standard-Port 1883 / TLS: 8883)
- Asynchrone Kommunikation über Broker
- OASIS-Standard (v3.1.1 und v5.0)

QoS-Level (Quality of Service)

- **QoS 0** —
At most once: keine Zustellgarantie, kein Acknowledge
- **QoS 1** —
At least once: garantierte Zustellung, Duplikate möglich
- **QoS 2** —
Exactly once: Vier-Wege-Handshake, höchste Verlässlichkeit

 Die Wahl des QoS-Levels ist ein direkter Trade-off zwischen Latenz, Netzwerklast und Verlässlichkeit.

MQTT: Broker-Konzept und Topic-Hierarchie

Broker-Funktion

Der Broker ist das zentrale Element im MQTT-Netz. Er empfängt alle Nachrichten von Publishern, filtert sie anhand des Topics und leitet sie an alle passenden Subscriber weiter. Die Kommunikationspartner kennen sich nicht direkt — vollständige Entkopplung.

- Persistenz via Retained Messages
- Session-Verwaltung über Clean Session-Flag
- Last Will & Testament für Verbindungsabbruch-Handling

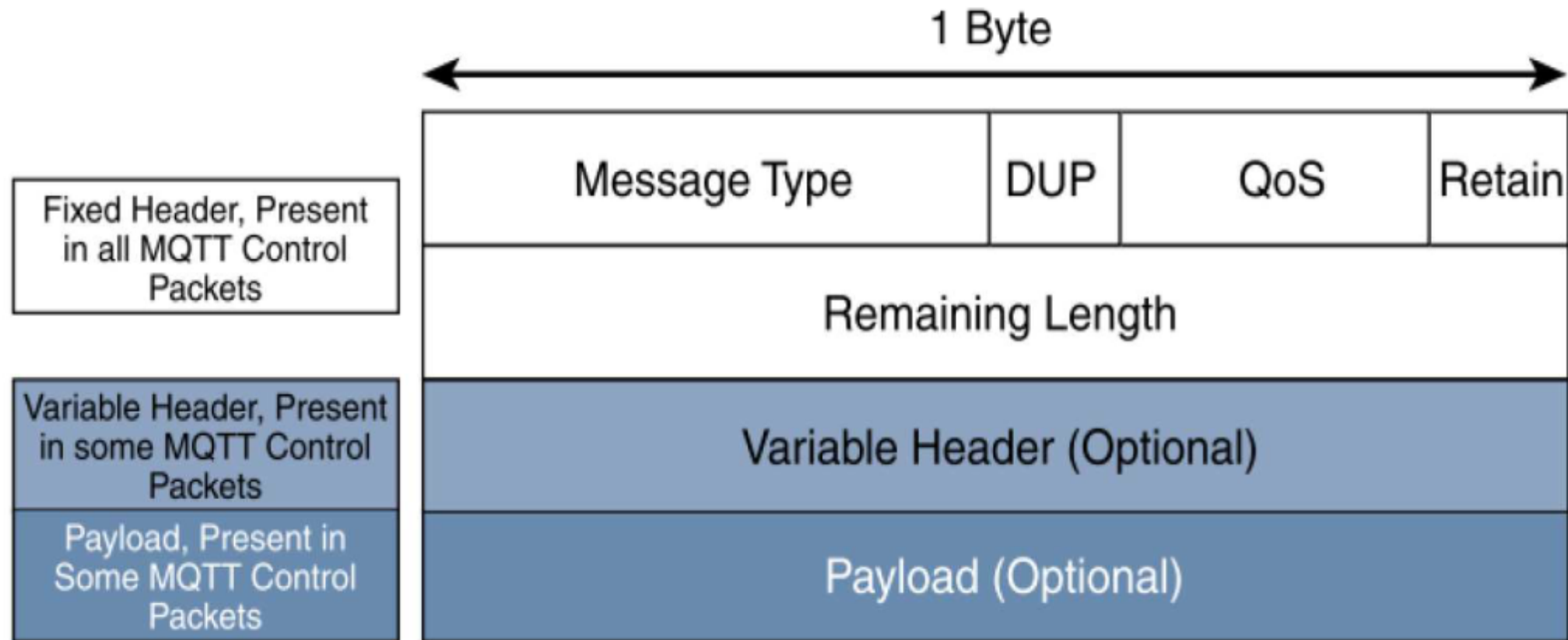
Topic-Hierarchie und Wildcards

Topics sind UTF-8-Strings mit / als Trenner. Wildcards ermöglichen flexible Subscription-Muster:

```
gebaeude/etage2/raum5/temperatur
gebaeude/+/raum5/temperatur    (Single Level)
gebaeude/#                     (Multi Level)
```

📄 Das #-Wildcard abonniert alle Topics unterhalb des angegebenen Levels — mit Vorsicht einzusetzen.

MQTT Nachrichtenaufbau



MQTT Nachrichtenarten

CONNECT / CONNACK: Verbindung aufbauen und bestätigen.

PUBLISH: eigentliche Nachricht mit Topic und Inhalt.

SUBSCRIBE / SUBACK: Topic abonnieren und bestätigen.

UNSUBSCRIBE / UNSUBACK: Abonnement beenden und bestätigen.

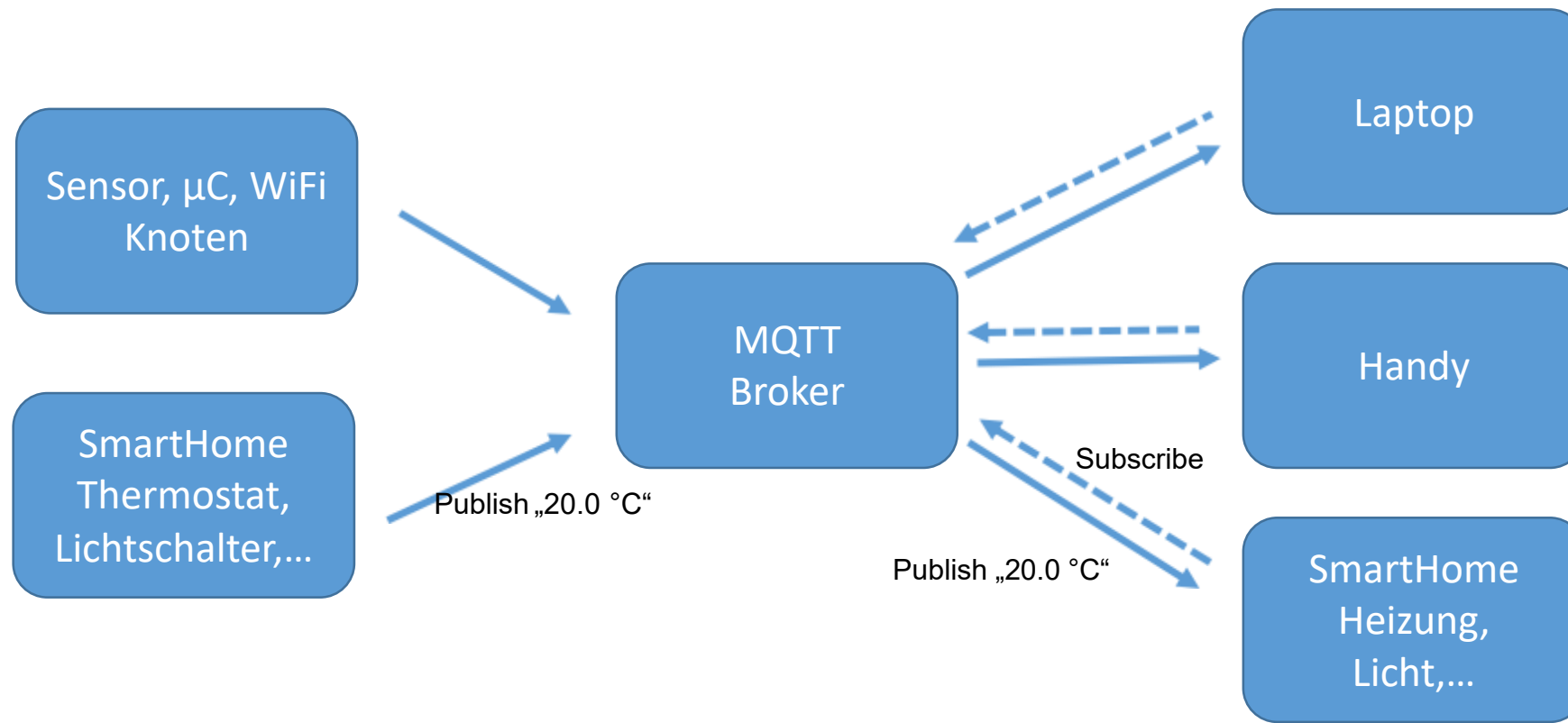
PINGREQ / PINGRESP: Verbindung am Leben halten.

DISCONNECT: sauber trennen.

PUBACK(nowledged), PUBREC(eived),
PUBREL(eased), PUBCOMP(lete): Bestätigungen für zuverlässige Zustellung bei QoS 1 und 2.

MQTT – Message Queue Telemetry Transport

MQTT: Publish/Subscribe



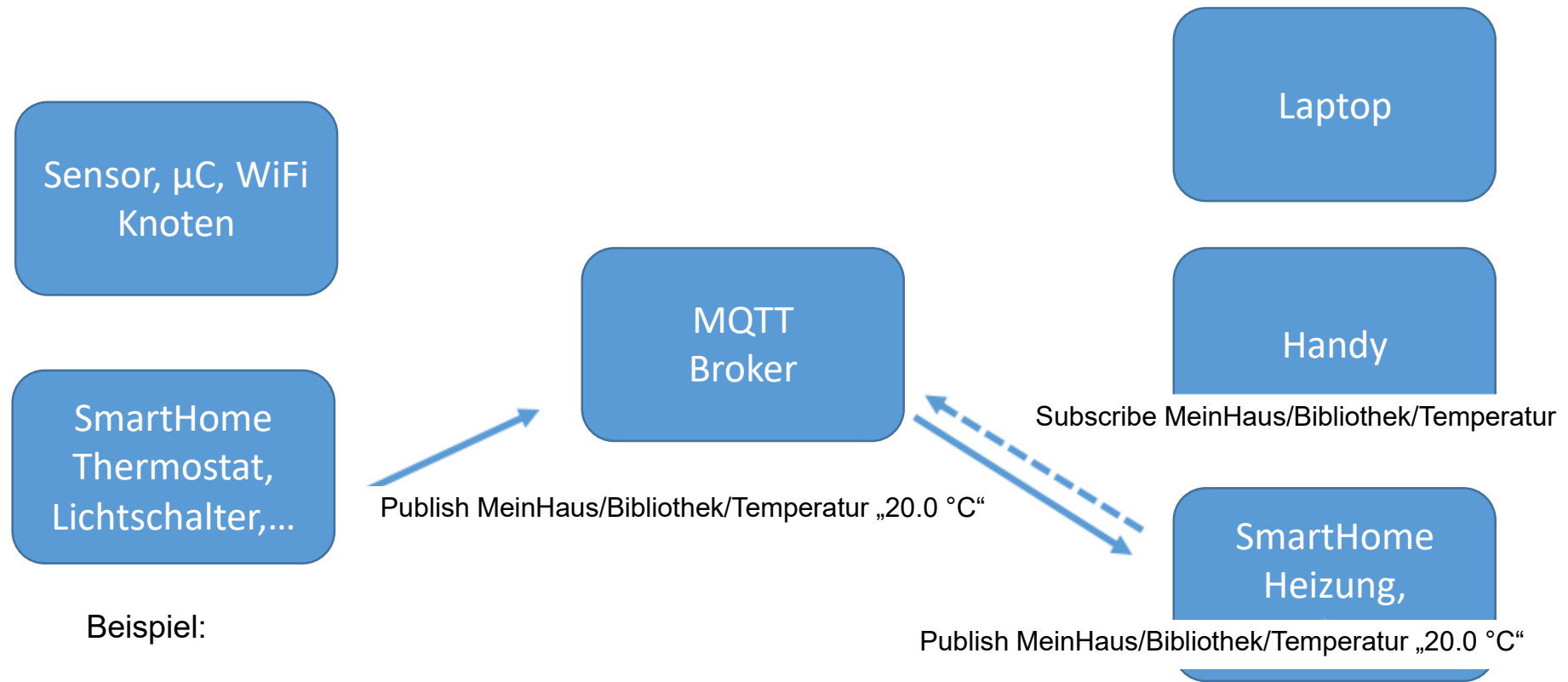
Im Gegensatz dazu HTTP: Request/Response

Bild nach: Walter Trokan, Das MQTT-Praxisbuch

MQTT – Message Queue Telemetry Transport

Adressierung über Topic („Betreff“)

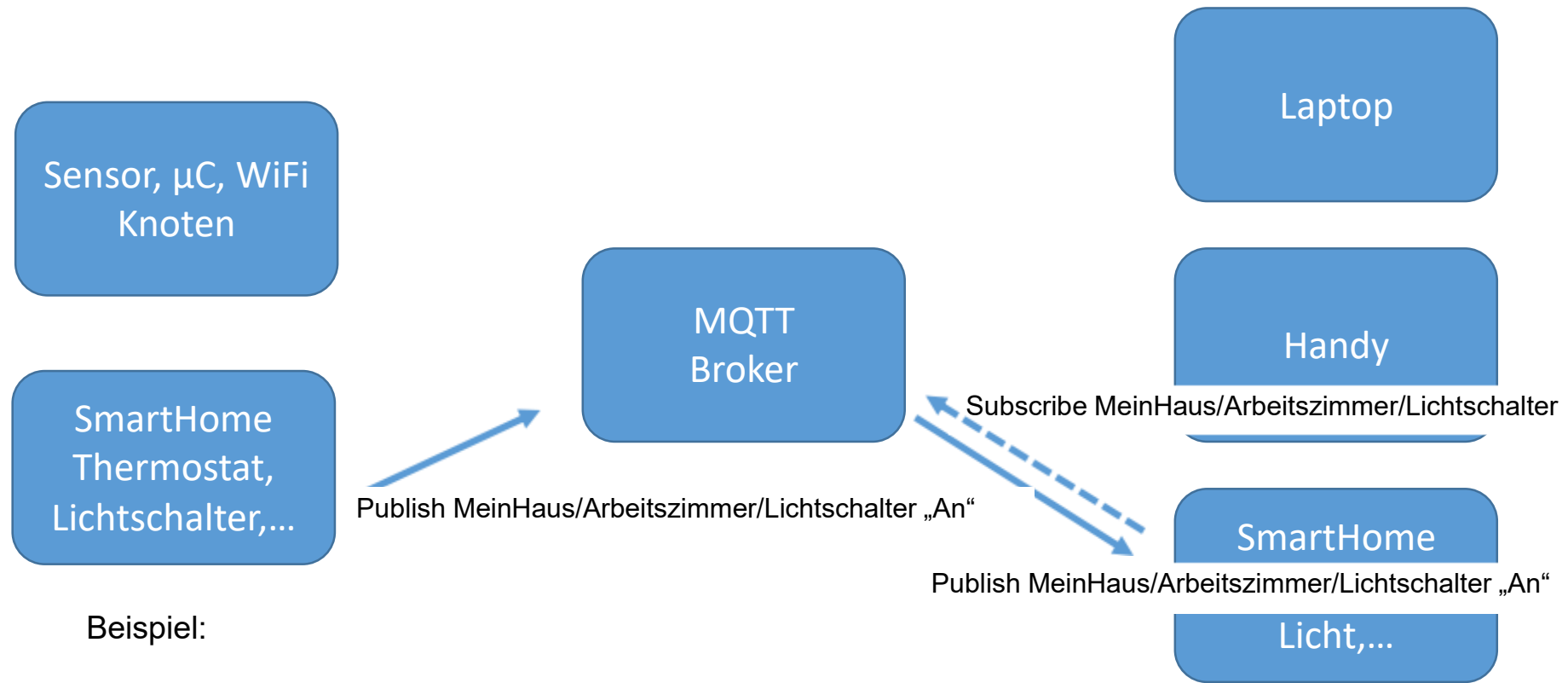
Nutzdaten als „Payload“



MQTT – Message Queue Telemetry Transport

Adressierung über Topic („Betreff“)

Nutzdaten als „Payload“

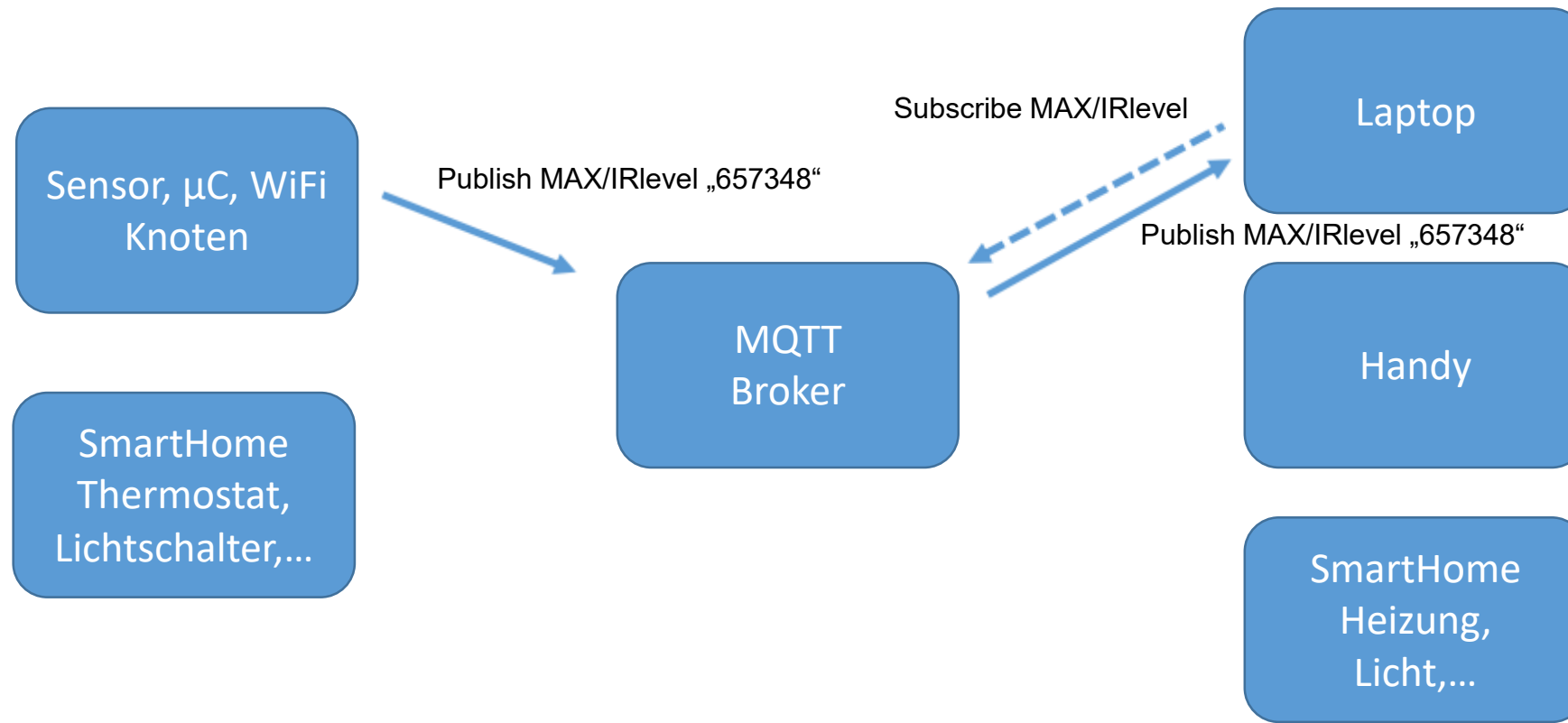


MQTT – Message Queue Telemetry Transport

„Wildcards“:

Single Level: MeinHaus/+/Temperatur

Multi Level: MeinHaus/#



MQTT – Qualitätssicherung

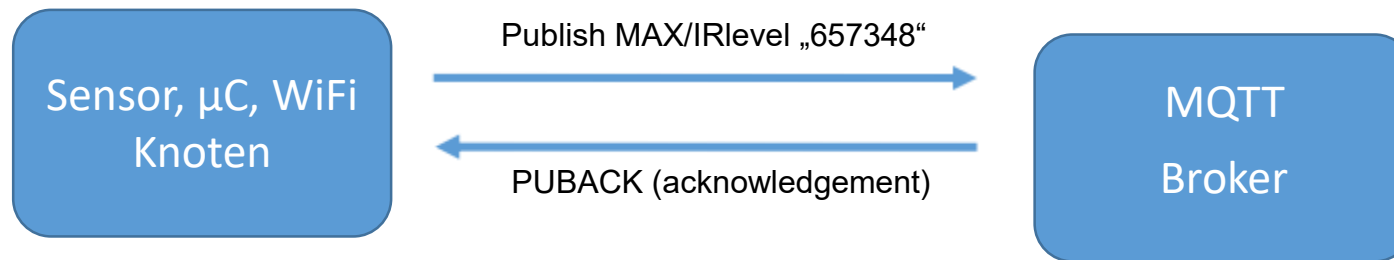
QoS0: Maximal einmal



Geringste Netzbelastung
Weder Empfangsbestätigung
noch Zwischenspeicherung beim Sender

MQTT – Qualitätssicherung

QoS1: Mindestens einmal

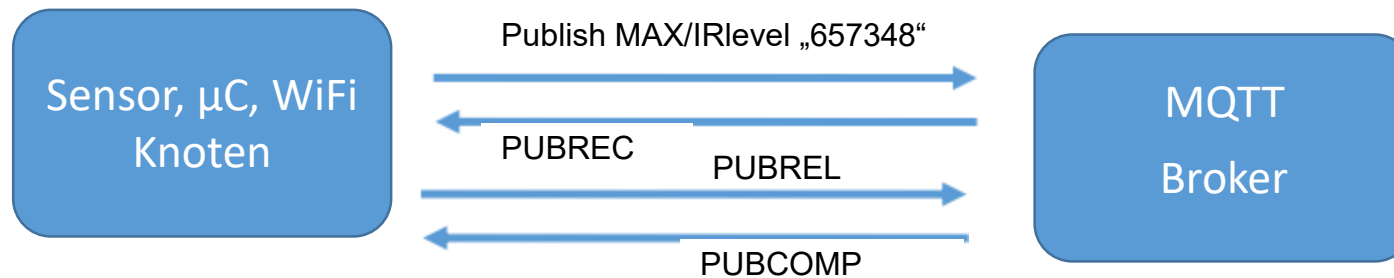


Moderate Netzbelastung
Empfangsbestätigung
Bis dahin Zwischenspeicherung beim Sender

Sender sendet bei ausbleibender Empfangsbestätigung mehrmals
Fehlermöglichkeit: z.B. Lichtschalter wird mehrmals betätigt

MQTT – Qualitätssicherung

QoS2: Garantiert nur einmal



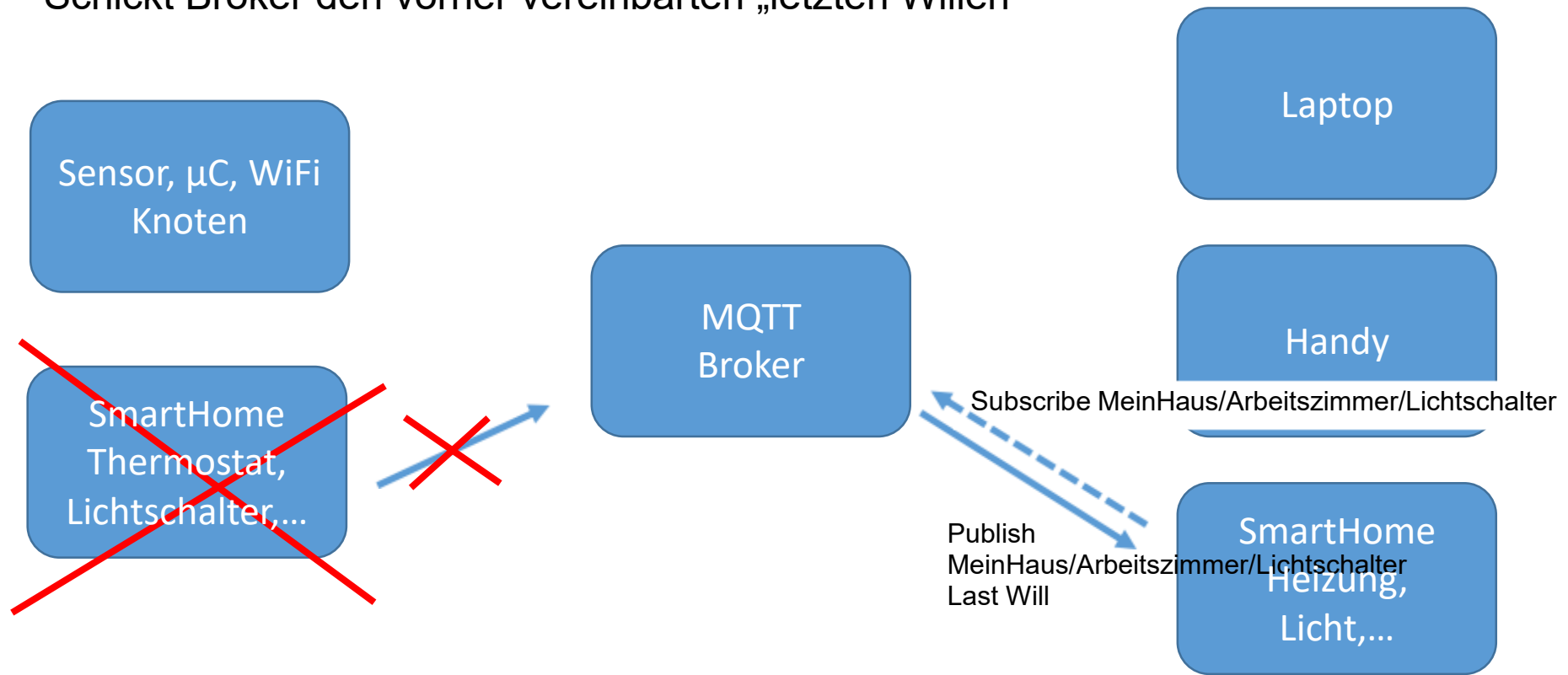
Moderate Netzbelastung
Empfangsbestätigung
Bis dahin Zwischenspeicherung beim Sender

Sender sendet bei ausbleibender Empfangsbestätigung mehrmals
Fehlermöglichkeit: z.B. Lichtschalter wird mehrmals betätigt

MQTT – LetzterWille/ Testament

Wenn Sensor „abnibbelt“

Schickt Broker den vorher vereinbarten „letzten Willen“



MQTT

Ausprobieren! Auf wokwi.com/esp32

Online ESP32 Simulator – MQTT Weather Logger

<http://www.hivemq.com/demos/websocket-client/>

Client Tools: MQTT-Explorer, MQTTX

Broker URL: broker.mqttdashboard.com

Port: 1883 (Ist der Default Port)

Subscription: `wokwi-weather`

➔ Save

WOKwi
World's most advanced ESP32 simulator

[Discord Community](#) [LinkedIn Group](#) [Pricing](#)

Online ESP32 Simulator

A faster way to prototype IoT projects with the ESP32 microcontrollers

Featured projects

- WiFi Scanning
- NTP Clock
- ESP32 HTTP Server
- Joke Machine
- MicroPython OLED Display
- MicroPython MQTT Weather Logger

Ausprobieren!:

Subscribe: wokwi-weather
Publish: wokwi-weather

„Hi, hier bin ich...“



Auf dem Dashboard nachsehen was angekommen ist

ESP32: WiFi Zugang

```
#include <WiFi.h>

const char* ssid = "Wokwi-GUEST";    // Dummy WLAN für IoT
const char* password = " ... ";

WiFiClient espClient;

void setup() {
    WiFi.begin(ssid, password);

    while(WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }

    Serial.println("WiFi connected");
    Serial.println("IP address: ");
    Serial.println(WiFi.localIP());
}
```

ESP32 : Druck-Sensor BMP085 auslesen

```
#include <Adafruit_BMP085.h>

Adafruit_BMP085 bmp;

WiFiClient espClient;
float pressure;

void setup() {

    bmp.begin();
}

void loop() {

    pressure = bmp.readPressure();

}
```


ESP32 : MQTT initialisieren

```
#include <PubSubClient.h>

const char* mqtt_server = "broker.mqttdashboard.com";

PubSubClient client(espClient);
char msg[75];

void callback(char* topic, byte* payload, unsigned int length) { ... }

void setup() {
    client.setServer(mqtt_server, 1883); // Serveradresse, Port
    client.setCallback(callback);
    client.connect("ESP32", "myuser", "mypass")
                // Nutzernamen, Account-Name, Passwort
    client.subscribe("wokwi-weather"); // Abonniertes Topic
}

void loop() {
    client.loop();

    if (!client.connected()) { reconnect(); }
```

ESP32 : MQTT initialisieren

Diese Funktion wird ausgeführt, wenn eine Message zu einem subskribierten Topic ankommt:

```
void callback(char* topic, byte* payload, unsigned int length) { ... }
```

```
void loop() {
```

```
  client.loop();
```

```
  if (!client.connected()) { reconnect(); }
```

```
}
```

Diese MQTT Funktion prüft, ob eine Message angekommen ist und ruft gegebenenfalls die als callback() eingetragene Funktion auf.



Regelmäßig (z.B. alle 10 s oder 1 min) sollte die Verbindung zum Broker überprüft werden. reconnect() ist eine selbst zu definierende Funktion, die die Verbindung (wie in setup() {...}) wieder aufbaut.



ESP32 : MQTT publish

```
#include <stdio.h>
```

```
snprintf(msg, 75, "%d", (int)(pressure);
```

```
client.publish(" Sensors /Pressure", msg);
```

Schreibt formatiert die integer Zahl aus der Variablen pressure in den String msg (wie printf(„%d“,pressure), hier Begrenzung auf n = 75 Bytes).



Publish den String aus dem Char Array msg unter dem Topic „UliFeder/Pressure“.



Für Fließkommazahlen muss man auf dem ESP32 (oder Arduino) tricksen, weil die abgespeckten Versionen von sprintf(...) etc. keine Fließkommaformate unterstützen:

```
snprintf(msg, 75, "%d.%02d", (int)(pressure/100), (int)pressure%100);
```

```
client.publish("Sensors/Pressure", msg);
```

ESP32 : MQTT subscribe

Einkommende Messages für ein subskribiertes Topic werden an die callback(...) Funktion weitergereicht und dort verarbeitet:

```
void callback(char* topic, byte* payload, unsigned int length) {  
    if ((char)payload[0] == 't') {  
        digitalWrite(BUILTIN_LED, LOW);    // Turn the LED on  
    } else {  
        digitalWrite(BUILTIN_LED, HIGH);   // Turn the LED off  
    }  
}
```

Hier wird (einfach) nur der erste Character msg[0] im msg Array mit „t“ verglichen. Ein „t“, „true“, „tobeornottobe“ schaltete die LED ein, ein „f“, „false“ aber auch ein „xbeliebig“ die LED aus.

MQTT – Wer holt die Daten ab?

Bis hier hin:

Der Sensor hat die Messdaten aufgenommen,
sich mit dem WLAN verbunden,
sich mit dem MQTT-Broker verbunden und
die Daten an Topics geschickt. (Können wir auf unserer App sehen)

Was kann man nun mit den Daten auf dem MQTT-Broker tun?
→ MQTT kann in eigene Programmen eingebunden werden,
um mit deren Hilfe die Messdaten darzustellen.

EIN Beispiel:

→ NodeRed

Alternativen zu MQTT

Je nach Anforderungsprofil — Bandbreite, Latenz, Skalierung, Nachrichtenformat — existieren etablierte Alternativen zu MQTT im IoT- und IIoT-Umfeld.

AMQP

Advanced Message Queuing Protocol. Vollständige Message-Queuing-Semantik mit Routing, Transaktionen und Persistenz. Höherer Overhead — für industrielle Backend-Kommunikation geeignet.

CoAP

Constrained Application Protocol (RFC 7252). UDP-basiert, Request/Response-Paradigma, optimiert für stark ressourcenbeschränkte Geräte. Unterstützt Observe-Extension für Push-Verhalten.

DDS

Data Distribution Service (OMG-Standard). Peer-to-Peer, hochperformant, echtzeitfähig. Eingesetzt in Robotik (ROS 2) und Verteidigung. Kein zentraler Broker.

Apache Kafka

Distributed Event Streaming Platform. Log-basiert, hoher Durchsatz, horizontale Skalierbarkeit. Nicht für ressourcenbeschränkte Geräte geeignet — ideal für IoT-Backend-Pipelines.

Node-RED: Flow-basierte Datenverarbeitung

Grundprinzip

Node-RED ist eine auf Node.js basierende, visuelle Programmierumgebung nach dem **Flow-Based Programming**-Paradigma (FBP). Verarbeitungslogik wird als gerichteter Graph modelliert: Knoten (Nodes) sind verbundene Funktionsbausteine, Kanten repräsentieren den Nachrichtenfluss.

- Nachrichtenobjekte: JSON-basierte `msg`-Objekte
- Ausführungsmodell: ereignisgesteuert, single-threaded Event Loop
- Browser-basierter Editor über HTTP

Node-Kategorien

- **Input Nodes:** MQTT-in, HTTP-in, WebSocket, Inject
- **Processing Nodes:** Function (JS), Switch, Change, Template
- **Output Nodes:** MQTT-out, HTTP-request, Debug, InfluxDB-out
- **Subflows:** Wiederverwendbare, gekapselte Teilgraphen



Node-RED-Flows werden als JSON-Dokument gespeichert und versionierbar in Git ablegt.

Node-RED: Einsatzgrenzen und Alternativen

Node-RED ist für prototypisches Entwickeln und mittlere Datenlast geeignet. Bei hohem Durchsatz, parallelen Verarbeitungspfaden oder strengen Produktionsanforderungen stoßen seine Grenzen an.

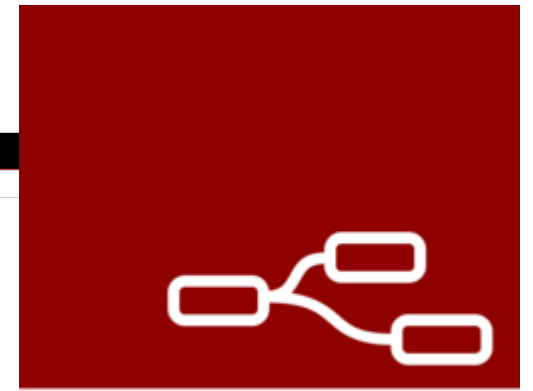
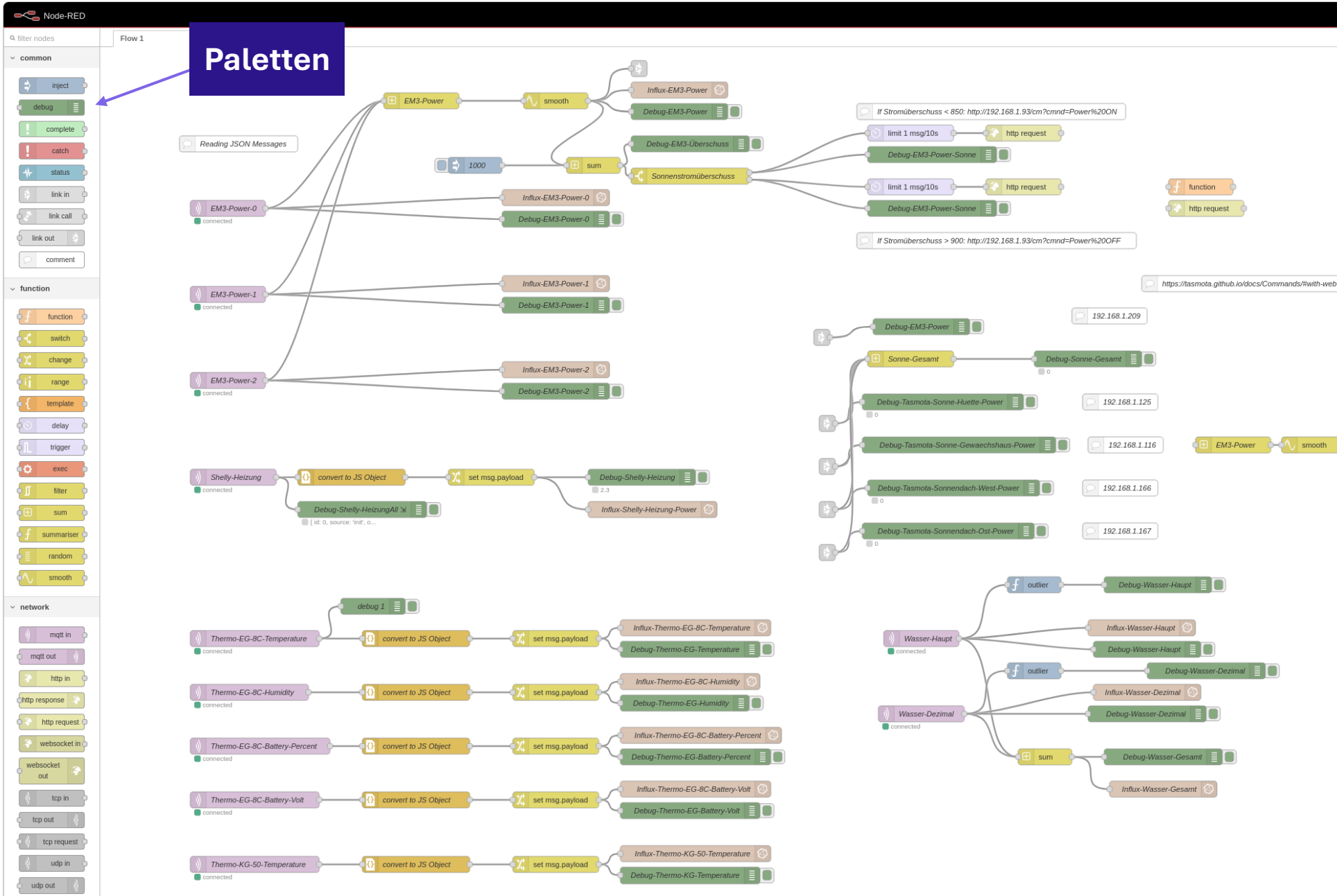
Technische Grenzen

- Single-threaded: kein echtes Parallelprocessing ohne Worker Threads
- Begrenzte Fehlertoleranz und kein nativer Clustering-Support
- Debugging komplexer Flows schwierig
- Versionskontrolle und CI/CD-Integration erfordern zusätzlichen Aufwand

Alternativen im Vergleich

- **Apache NiFi**: Enterprise-Grade, Backpressure, Provenance Tracking, hoher Ressourcenbedarf
- **Apache Kafka Streams / ksqlDB**: Stream-Processing mit SQL-Semantik, horizontal skalierbar
- **AWS IoT Greengrass / Azure IoT Edge**: Cloud-native Edge-Verarbeitung, Managed Lifecycle
- **n8n**: Open-Source, workflow-orientiert, stärkerer Fokus auf Integrationsszenarien

Node Red



Node-RED

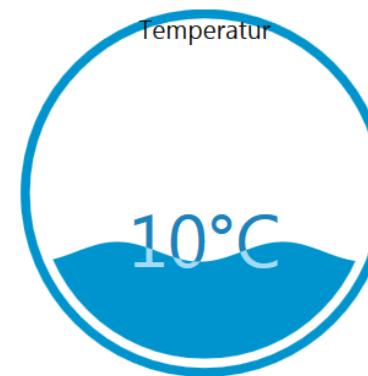
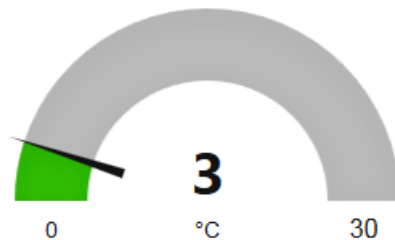
Node-Red

Flussdiagramm-basiertes Programmierwerkzeug (flow-based programming tool)

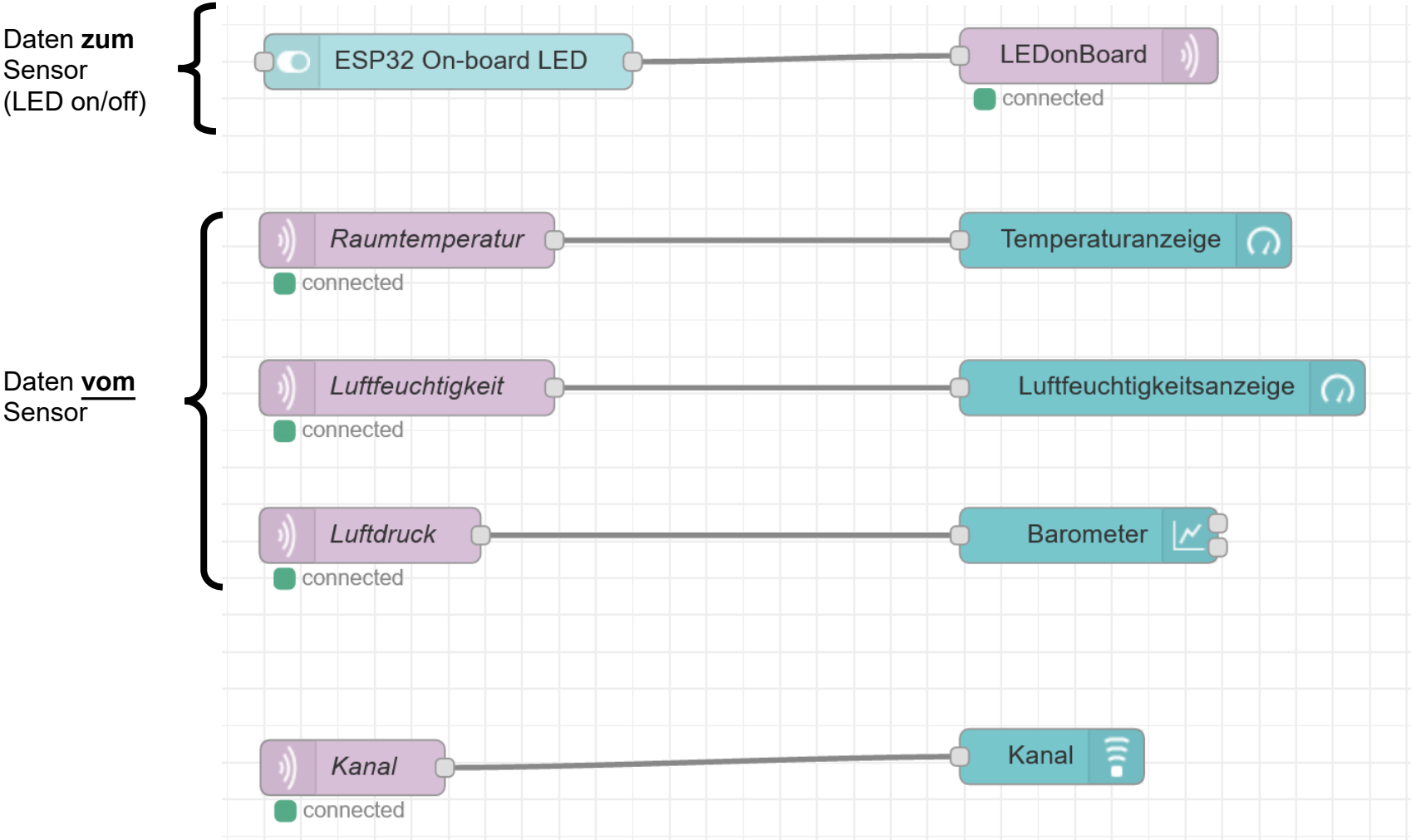


Es wird von einer Laufzeitumgebung (Programm Node-RED) ein „Dashboard“ erzeugt, das über einen Browser angezeigt werden kann:

aktuelle Temperatur in Chemnitz

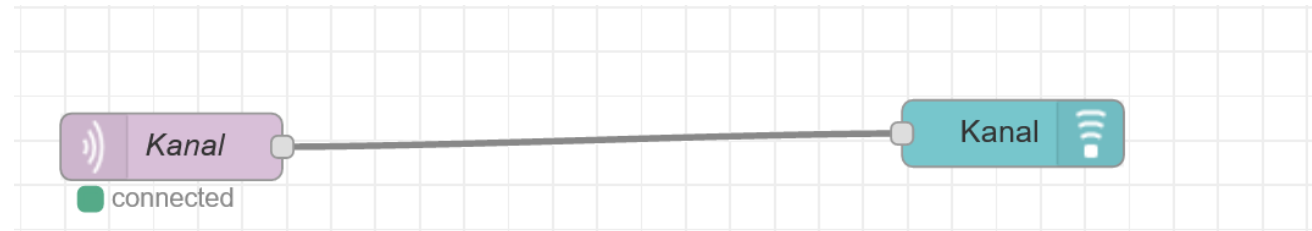


Node-Red



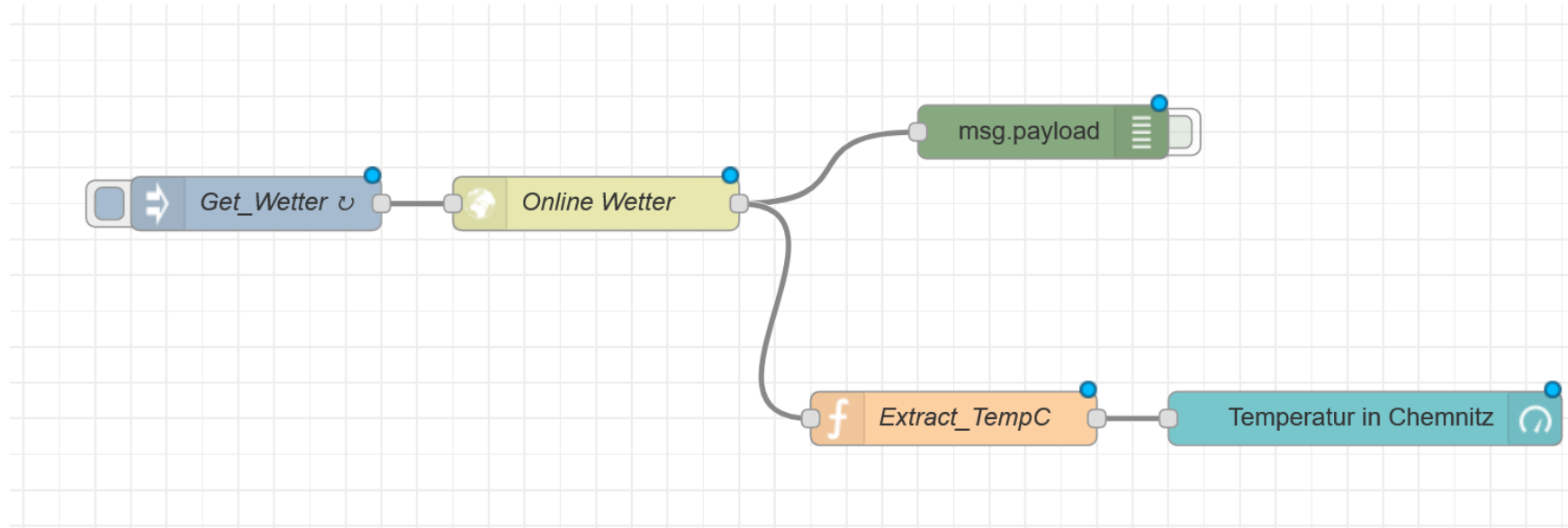
Node-Red

TTS: Text-to-speech



Alle Nachrichten an das Topic „myTopic/Kanal“ werden durch den Lautsprecher als Sprache ausgegeben.
(solange man das NodeRed-Dashboard geöffnet hat)

Node-Red



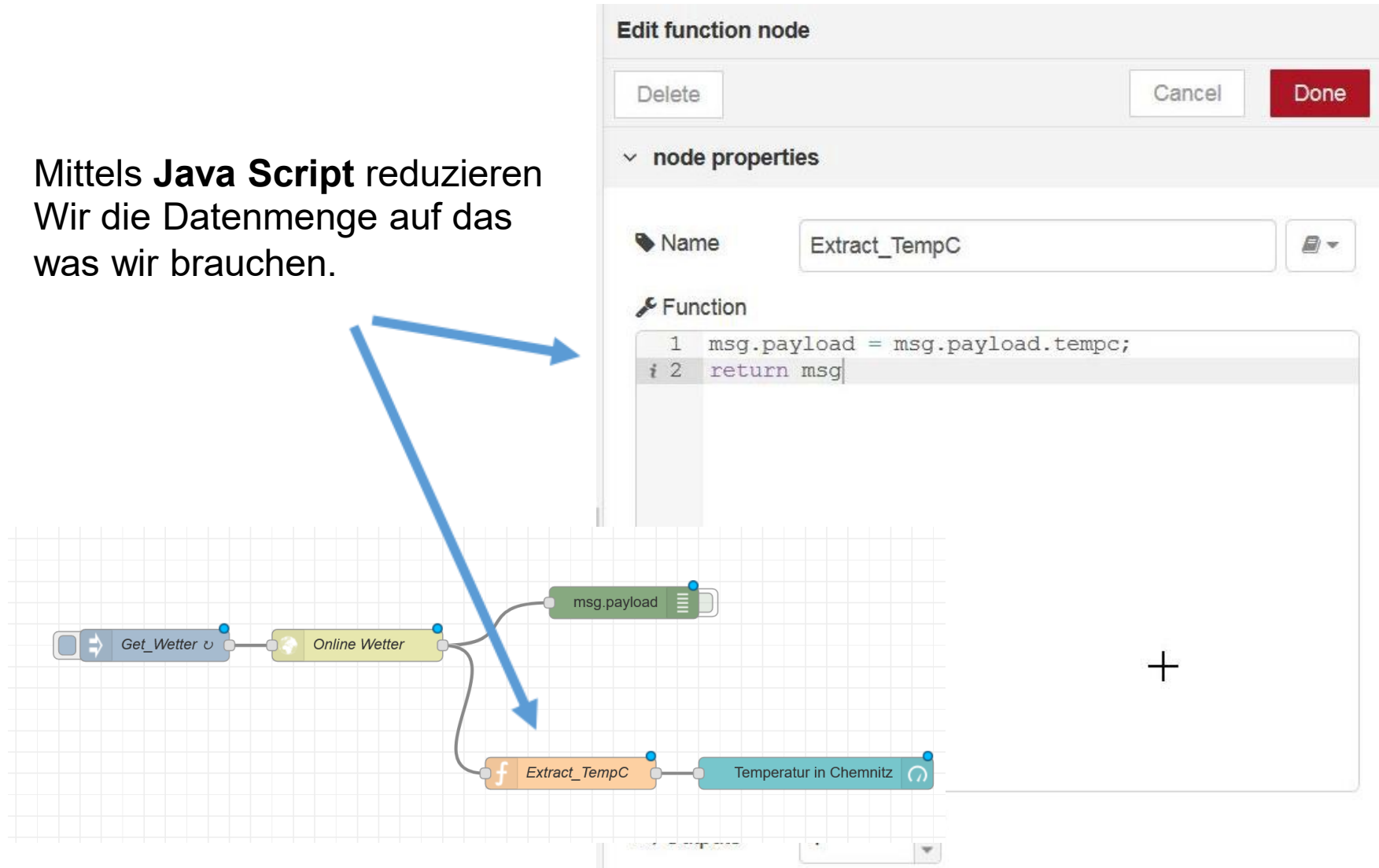
Alle 10 Sekunden aktiviert die Node „Get_Wetter“ die nachfolgenden Node „Online Wetter“, welche auf die API von OpenWeatherMap zugreift. Wir erhalten dadurch eine große Menge an Daten aus der API

Wetterdaten:

<https://home.openweathermap.org>

Node-Red

Mittels **Java Script** reduzieren
Wir die Datenmenge auf das
was wir brauchen.



Payload - msg

Was für Daten könnten wir alles bekommen?

Number

String

csv

Html

xml

wav

Video

JSON Objekt

→ häufig genutztes Format

The screenshot shows the Node-RED interface with the 'debug' tab selected. The console displays a message from node 'a4c27bd5.32247' at 11/01/2018, 22:39:20. The payload is a JSON object representing weather data for Chemnitz. The object includes fields for weather type (Clouds), detail (Überwiegend bewölkt), icon (04n), temperatures (tempk: 276.15, tempc: 3, temp_maxc: 3, temp_minc: 3), humidity (93), maxtemp (276.15), mintemp (276.15), windspeed (0.5), location (Chemnitz), sunrise (1515654399), sunset (1515684405), and clouds (75). A description is also provided: 'The weather in Chemnitz at coordinates: 50.83, 12.93 is Clouds (Überwiegend bewölkt).'. Below this, another message from node '379ef5e9.e734fa' is partially visible, showing a payload of the number 3.

```
3
11/01/2018, 22:39:20 node: a4c27bd5.32247
Wie_ist_das_Wetter : msg.payload : Object
▼ object
  weather: "Clouds"
  detail: "Überwiegend bewölkt"
  icon: "04n"
  tempk: 276.15
  tempc: 3
  temp_maxc: 3
  temp_minc: 3
  humidity: 93
  maxtemp: 276.15
  mintemp: 276.15
  windspeed: 0.5
  location: "Chemnitz"
  sunrise: 1515654399
  sunset: 1515684405
  clouds: 75
  description: "The weather in Chemnitz at coordinates: 50.83, 12.93 is Clouds (Überwiegend bewölkt)."
```

```
11/01/2018, 22:39:20 node: 379ef5e9.e734fa
Wie_ist_das_Wetter : msg.payload : number
3
```

```
11/01/2018, 22:39:30 node: a4c27bd5.32247
```

JSON – JavaScript Object Notation

Ausgabe aus OpenWeatherMap:

```
{"weather":"Clouds","detail":"Überwiegend bewölkt","icon":"04n","tempk":276.15,"tempc":3,"temp_maxc":3,"temp_minc":3,"humidity":93,"maxtemp":276.15,"mintemp":276.15,"windspeed":0.5,"location":"Chemnitz","sunrise":1515654399,"sunset":1515684405,"clouds":75,"description":"The weather in Chemnitz at coordinates: 50.83, 12.93 is Clouds (Überwiegend bewölkt)."}}
```



JSON – JavaScript Object Notation

```
{
  "weather": "Clouds",
  "detail": "Überwiegend bewölkt",
  "icon": "04n",
  "tempk": 276.15,
  "tempc": 3,
  "temp_maxc": 3,
  "temp_minc": 3,
  "humidity": 93,
  "maxtemp": 276.15,
  "mintemp": 276.15,
  "windspeed": 0.5,
  "location": "Chemnitz",
  "sunrise": 1515654399,
  "sunset": 1515684405,
  "clouds": 75,
  "description": "The weather in Chemnitz at coordinates: 50.83, 12.93 is Clouds (Überwiegend bewölkt)."}
}
```

The screenshot shows the Node-RED web interface. At the top, there are tabs for 'info', 'debug', and 'dashboard'. The 'debug' tab is selected. Below the tabs, there is a search bar with 'all nodes' and a trash icon. The main area displays a list of nodes. The first node is selected, showing its details. The node is a 'msg.payload' object. The payload is a JSON object with the following properties: weather: "Clouds", detail: "Überwiegend bewölkt", icon: "04n", tempk: 276.15, tempc: 3, temp_maxc: 3, temp_minc: 3, humidity: 93, maxtemp: 276.15, mintemp: 276.15, windspeed: 0.5, location: "Chemnitz", sunrise: 1515654399, sunset: 1515684405, clouds: 75, and description: "The weather in Chemnitz at coordinates: 50.83, 12.93 is Clouds (Überwiegend bewölkt).". The node is labeled 'Wie_ist_das_Wetter : msg.payload : Object'.

JSON – JavaScript Object Notation

... is a lightweight data-interchange format.

... completely language independent but uses conventions that are familiar to programmers of the C-family of languages, including C, C++, C#, Java, JavaScript, Perl, Python, and many others.

A collection of name/value pairs (objects).

An ordered list of values (arrays).

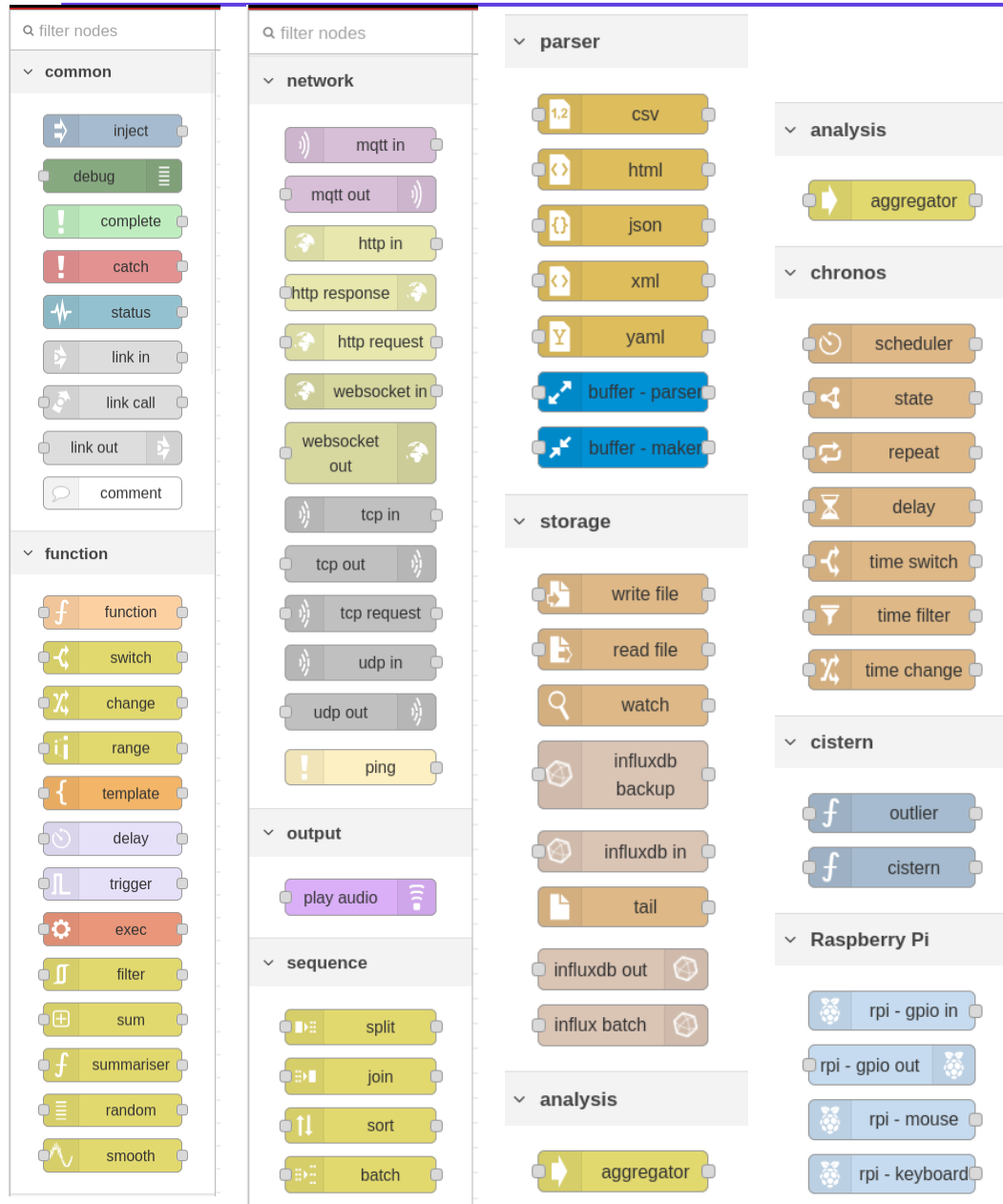
(Zitate aus <https://www.json.org/>)

Beispiel „Kreditkarte“

(aus https://de.wikipedia.org/wiki/JavaScript_Object_Notation)

```
{
  "Herausgeber": "Xema",
  "Nummer": "1234-5678-9012-3456",
  "Deckung": 2e+6,
  "Waehrung": "EURO",
  "Inhaber":
  {
    "Name": "Mustermann",
    "Vorname": "Max",
    "maennlich": true,
    "Hobbys": ["Reiten", "Golfen", "Lesen"],
    "Alter": 42,
    "Kinder": [],
    "Partner": null
  }
}
```

Node Red ist erweiterbar über Paletten



Node-RED: Nodes & Paletten

Nodes

- Bausteine für Datenverarbeitung: Input, Output, Function etc.
- Per Drag-and-Drop zu Flows verbunden.

Paletten

- Linke Seitenleiste mit Core-Nodes (ca. 40).
- Filterbar und durchsuchbar.

Erweiterbarkeit

- Tausende Community-Nodes via Palette Manager installierbar.
- Eigene Nodes entwickelbar.

Fazit

- Flexibles Low-Code-Tool für IoT-Automatisierung dank visueller Flows und hoher Erweiterbarkeit.



InfluxDB: Zeitreihendatenbanken

InfluxDB ist eine spezialisierte **Time Series Database (TSDB)**, optimiert für hochfrequente, zeitgestempelte Messpunkte — das typische Datenprofil von IoT-Sensorik.

Datenmodell (InfluxDB v2/v3)

- **Measurement:** Tabelle / Messkategorie (z.B. temperature)
- **Tags:** Indizierte Metadaten (String), für Filterung und Grouping
- **Fields:** Messwerte (Float, Int, String, Bool), nicht indiziert
- **Timestamp:** Nanosekunden-Präzision, UTC

☐ Tags sind indiziert und für Abfragen zu bevorzugen. Fields enthalten die eigentlichen Messwerte.

Abfragesprachen

- **Flux (v2):** Funktionale Pipelinesprache, ausdrucksstark, lernintensiv
- **InfluxQL (v1):** SQL-ähnlich, einfacher Einstieg, geringere Ausdrucksmächtigkeit
- **SQL (v3/IOx):** Apache Arrow-basiert, vollständige SQL-Unterstützung geplant

```
from(bucket: "iot")
  |> range(start: -1h)
  |> filter(fn: (r) => r._measurement == "temperature")
  |> mean()
```

InfluxDB v1 und InfluxQL: Die klassische Abfragesprache

InfluxDB Version 1 war die erste weit verbreitete Version der Zeitreihendatenbank und nutzte InfluxQL als primäre Abfragesprache. Eine SQL-ähnliche Sprache, die speziell für Zeitreihendaten entwickelt wurde.

InfluxDB v1 — Überblick

- Erschienen: 2013, stabile v1.x-Serie ab 2016
- Datenmodell: Measurements, Tags, Fields, Timestamps
- Speicher-Engine: TSM (Time-Structured Merge Tree)
- Retention Policies: Automatisches Ablaufen alter Daten
- Continuous Queries: Automatisches Downsampling
- Datenbank-Konzept: Mehrere Datenbanken pro Instanz
- Schnittstellen: HTTP API, Line Protocol für Schreiboperationen
- Lizenz: MIT (Open Source, vollständig frei)

InfluxQL — Syntax und Beispiele

InfluxQL orientiert sich stark an SQL, ist aber auf Zeitreihensemantik zugeschnitten. Wichtige Unterschiede: kein JOIN, kein GROUP BY über mehrere Measurements.

```
-- Einfache Abfrage
SELECT mean("value") FROM "temperature"
WHERE time > now() - 1h
GROUP BY time(5m), "location"

-- Mit Retention Policy
SELECT * FROM "autogen"."cpu_load"
WHERE "host" = 'server01'
ORDER BY time DESC LIMIT 100

-- Continuous Query (Downsampling)
CREATE CONTINUOUS QUERY "cq_1h"
ON "iot_db"
BEGIN
    SELECT mean("value") INTO "downsampled"."temperature"
    FROM "temperature" GROUP BY time(1h)
END
```

 InfluxQL ist in InfluxDB v1 und v2 (Kompatibilitätsmodus) verfügbar. Ab v2 wurde Flux als primäre Sprache eingeführt; ab v3 (IOx) wird SQL bevorzugt. InfluxQL bleibt für Legacy-Systeme relevant.

InfluxDB: Speicheroptimierung und Alternativen

Speicher- und Performanceeigenschaften

- **TSM-Engine** (Time-Structured Merge Tree): Optimiert für sequentielles Schreiben
- Automatische Kompression von Zeitreihendaten
- **Retention Policies**: Automatisches Löschen alter Daten nach definierter Dauer
- **Continuous Queries / Tasks**: Downsampling für langfristige Aggregation
- Hohe Schreibrate: Millionen von Datenpunkten pro Sekunde möglich

Alternativen zu InfluxDB

- **TimescaleDB**: PostgreSQL-Erweiterung für Zeitreihen — SQL-Kompatibilität, relationale Features
- **Prometheus**: Pull-basiert, Fokus auf Monitoring-Metriken, eingebaute Alerting-Engine
- **OpenTSDB**: HBase-basiert, für sehr große Cluster geeignet
- **QuestDB**: Hochperformant, SQL-basiert, geringer Ressourcenverbrauch
- **Apache Cassandra**: Wide-Column Store, geeignet für verteilte IoT-Historisierung

Grafana: Visualisierung und Monitoring



Grafana ist eine Open-Source-Plattform zur Visualisierung von Zeitreihendaten aus heterogenen Quellen. Es fungiert als universelles **Observability-Frontend** und ist vollständig von der Datenhaltung entkoppelt.



Dashboards

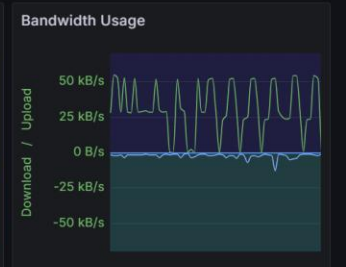
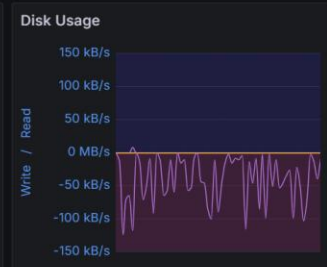
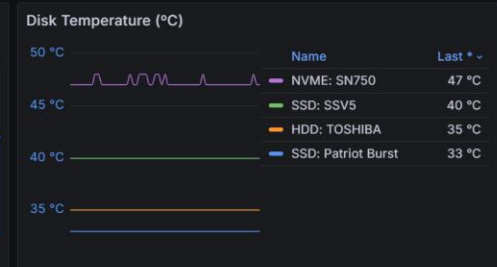
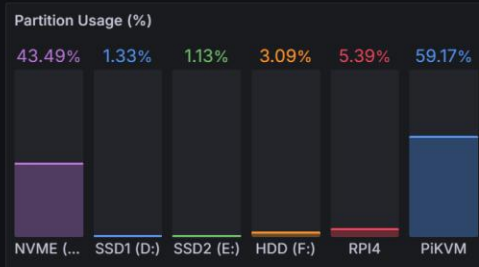
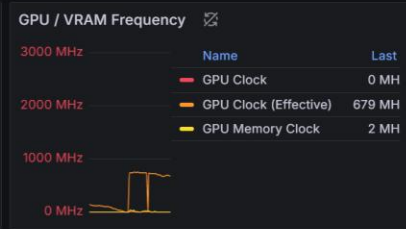
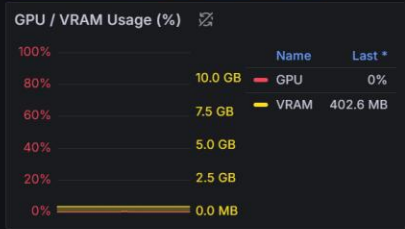
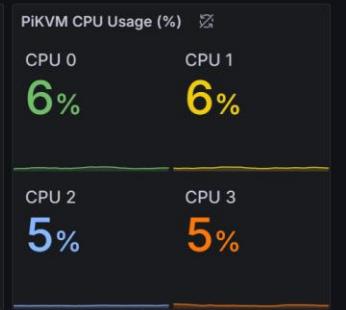
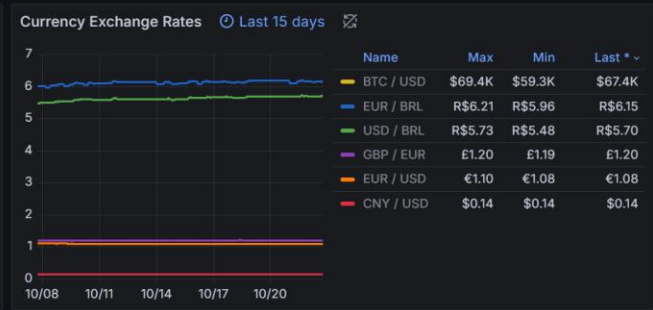
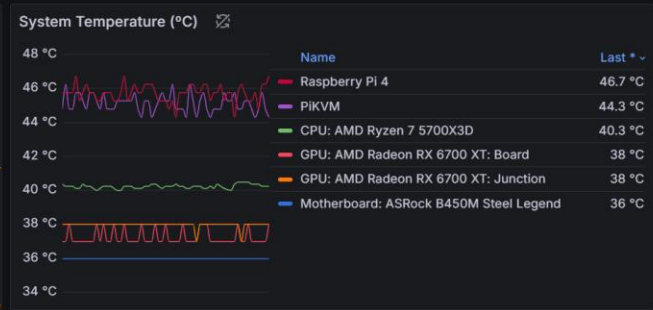
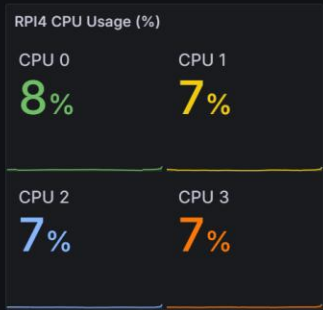
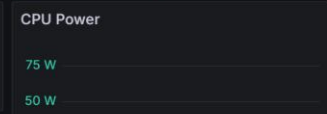
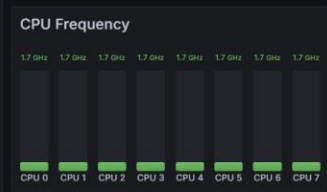
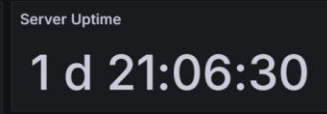
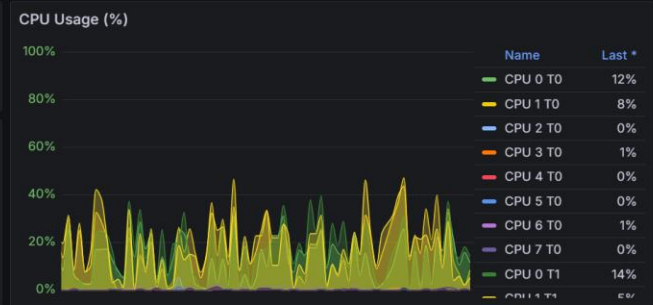
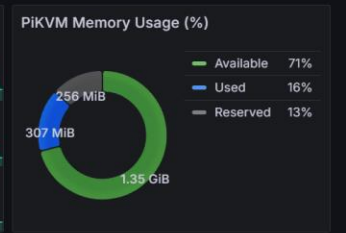
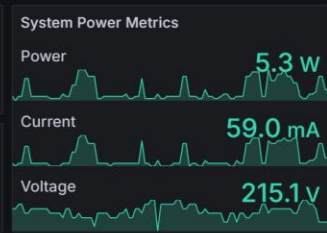
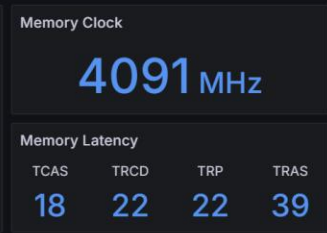
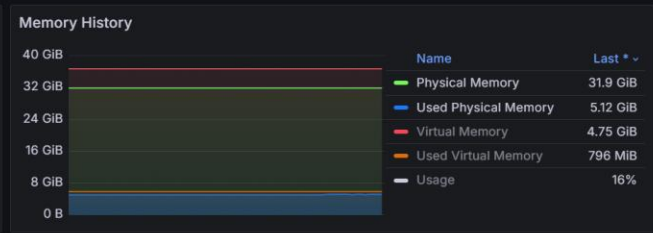
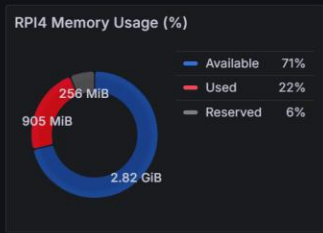
Panels aus verschiedenen Visualisierungstypen: Zeitreihen, Gauges, Heatmaps, Tabellen, Geomap

Datenquellen

Native Plugins für InfluxDB, Prometheus, Elasticsearch, PostgreSQL, Loki u.v.m.

Alerting

Regelbasierte Schwellwertüberwachung, Notifications via E-Mail, Slack, PagerDuty, Webhooks



Grafana: Templating, Annotations und Alternativen

Erweiterte Funktionen

- **Template Variables:** Dynamische Dashboards — Gerät, Standort oder Zeitraum als Parameter
- **Annotations:** Ereignismarkierungen im Zeitstrahl (z.B. Deployments, Alarme)
- **Grafana Loki:** Log-Aggregation als komplementäre Komponente
- **Grafana Tempo:** Distributed Tracing-Integration
- **Grafana Alloy:** OpenTelemetry-kompatibler Collector

Alternativen zu Grafana

- **Kibana:** Bestandteil des ELK-Stacks, stark auf Log-Analyse fokussiert
 - **Elastic Stack:** (ELK) Elasticsearch, Kibana & Logstash
- **Chronograf:** Natives InfluxDB-Frontend, geringerer Funktionsumfang
- **Apache Superset:** SQL-first BI-Tool, breite Datenbankunterstützung
- **Metabase:** Einfacher Einstieg, Self-Service BI, weniger für Echtzeit-Monitoring
- **Datadog / Dynatrace:** SaaS-Monitoring-Plattformen, Vollintegration, kostenpflichtig

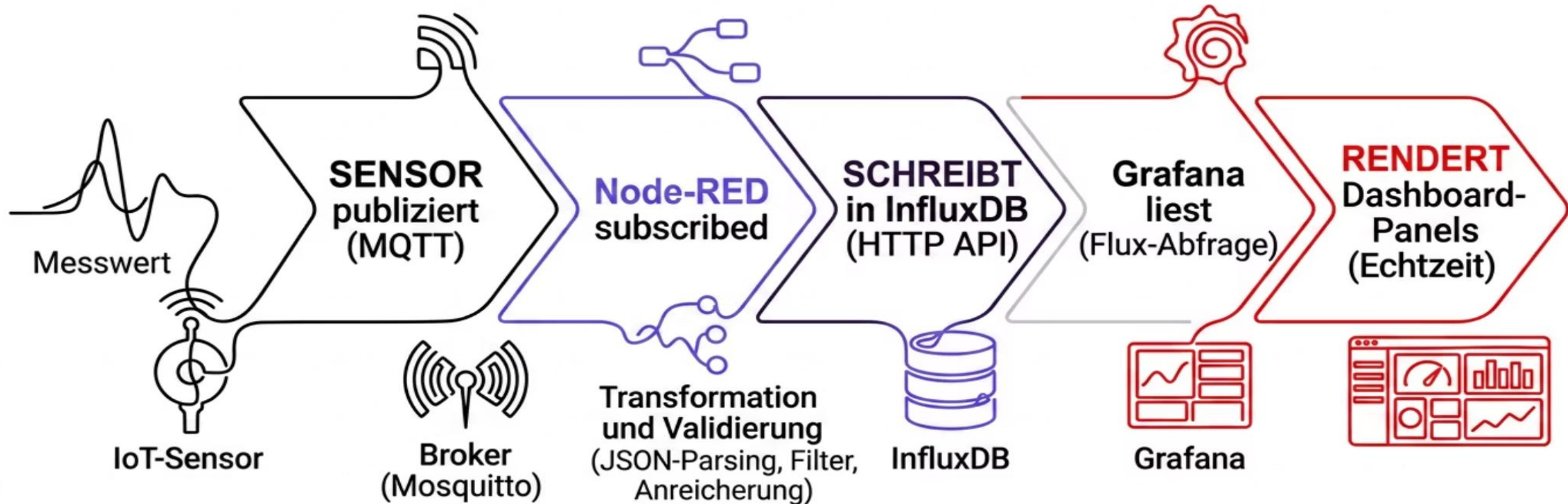
Technologievergleich: MING vs. Alternativen

Die folgende Tabelle fasst die Kernkomponenten des MING-Stacks und ihre wichtigsten Alternativen nach Einsatzszenario zusammen.

Schicht	MING-Komponente	Alternative(n)	Typisches Entscheidungskriterium
Transport	MQTT	AMQP, CoAP, DDS, Kafka	Gerätekategorie, Latenz, Skalierung, QoS-Anforderung
Verarbeitung	Node-RED	Apache NiFi, Kafka Streams, AWS IoT Edge	Datendurchsatz, Produktionsreife, Komplexität
Persistenz	InfluxDB	TimescaleDB, Prometheus, QuestDB	Abfragesprache, SQL-Kompatibilität, Skalierbarkeit
Visualisierung	Grafana	Kibana, Superset, Datadog	Datenquellenvielfalt, Alerting, Managed vs. Self-hosted

Systemintegration: End-to-End-Datenfluss

Ein typischer IoT-Datenpfad im MING-Stack folgt einer klaren, sequenziellen Verarbeitungskette vom Sensor bis zum Dashboard — mit definierten Schnittstellen zwischen den Komponenten.



Jede Komponentengrenze ist eine potenzielle Integrationsschicht: Der Datenaustausch erfolgt über standardisierte Protokolle (MQTT, HTTP/REST, Line Protocol), was den Stack modular und wartbar hält.

Zusammenfassung und Einordnung

Stärken des MING-Stacks

- Vollständig Open Source, aktive Communities
- Modulare Architektur: jede Komponente unabhängig austauschbar
- Hohe Integration: native Plugins zwischen allen Schichten
- Leichtgewichtig: geeignet für Edge-Deployments und Raspberry Pi
- Breite Industrie-Akzeptanz in IIoT und Smart Building

Grenzen und Trade-offs

- Node-RED: nicht für hohe Last ohne Anpassung geeignet
- InfluxDB: Lizenzmodell seit v2 teils restriktiv (BSL)
- Kein natives Clustering oder Hochverfügbarkeitskonzept out-of-the-box
- Sicherheit (TLS, Auth) muss explizit konfiguriert werden

Empfehlung

Der MING-Stack ist als **Referenzarchitektur für IoT-Prototypen und mittlere Produktionsszenarien** geeignet. Für Enterprise-Skala oder Cloud-native Anforderungen sind hybride oder alternative Stacks zu evaluieren.

OpenHAB: open Home Automation Bus

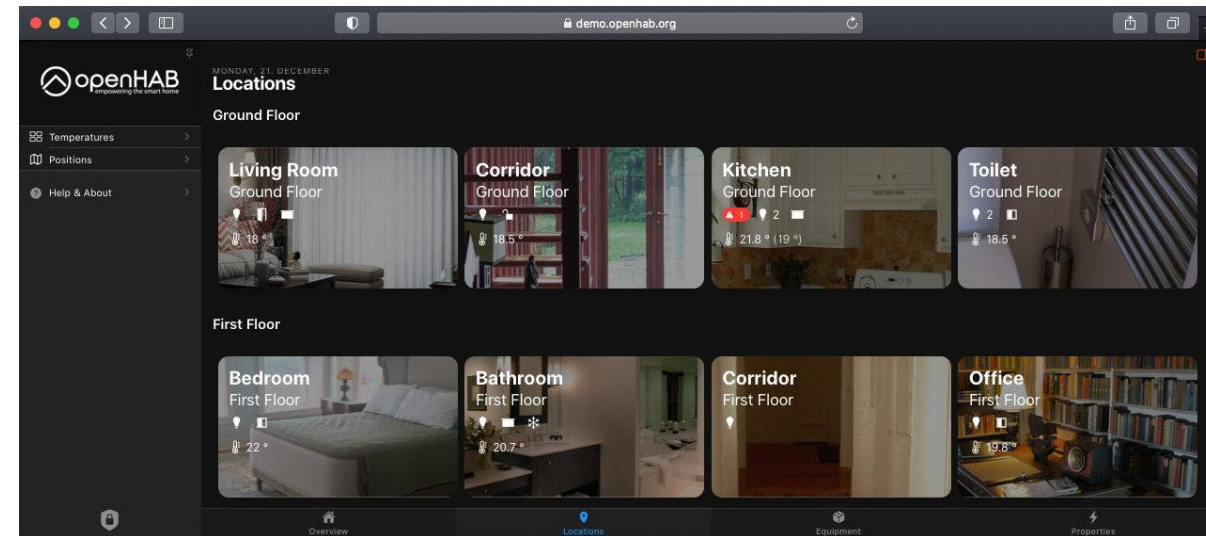
<http://www.openhab.org/>

Automatisierungssoftware für das „Smart Home“

Steuert (Licht, Heizung, Stereoanlage, Waschmaschine, Katze,...)

Bildet Strukturen ab

- Keller, Erdgeschoss, Gartenlaube,...
- Wohnzimmer, Küche, Bibliothek,...
- Heizung, Licht, Kochen, Unterhaltung,...



Speichert Zustände (Licht an, Temperatur im Wohnzimmer,...)

- Regelt (am Wochenende eine Stunde vor Sonnenaufgang das Bad heizen und die Katze füttern)

Viele „Bindings“ für Geräte von unterschiedlichen Herstellern nach unterschiedlichen Normen (KNX, ENOCEAN, HomeMatic, ...)

Visuelle Programmierung

The screenshot displays the openHAB web interface at demo.openhab.org. The left sidebar contains navigation links for Temperatures, Positions, Administration (Settings, Things, Model, Items, Pages, Rules), Scripts (selected), Schedule, Developer Tools, and Help & About. The main workspace is titled 'Simple Per-Room Climate Control' and shows a visual programming script for an application/javascript.

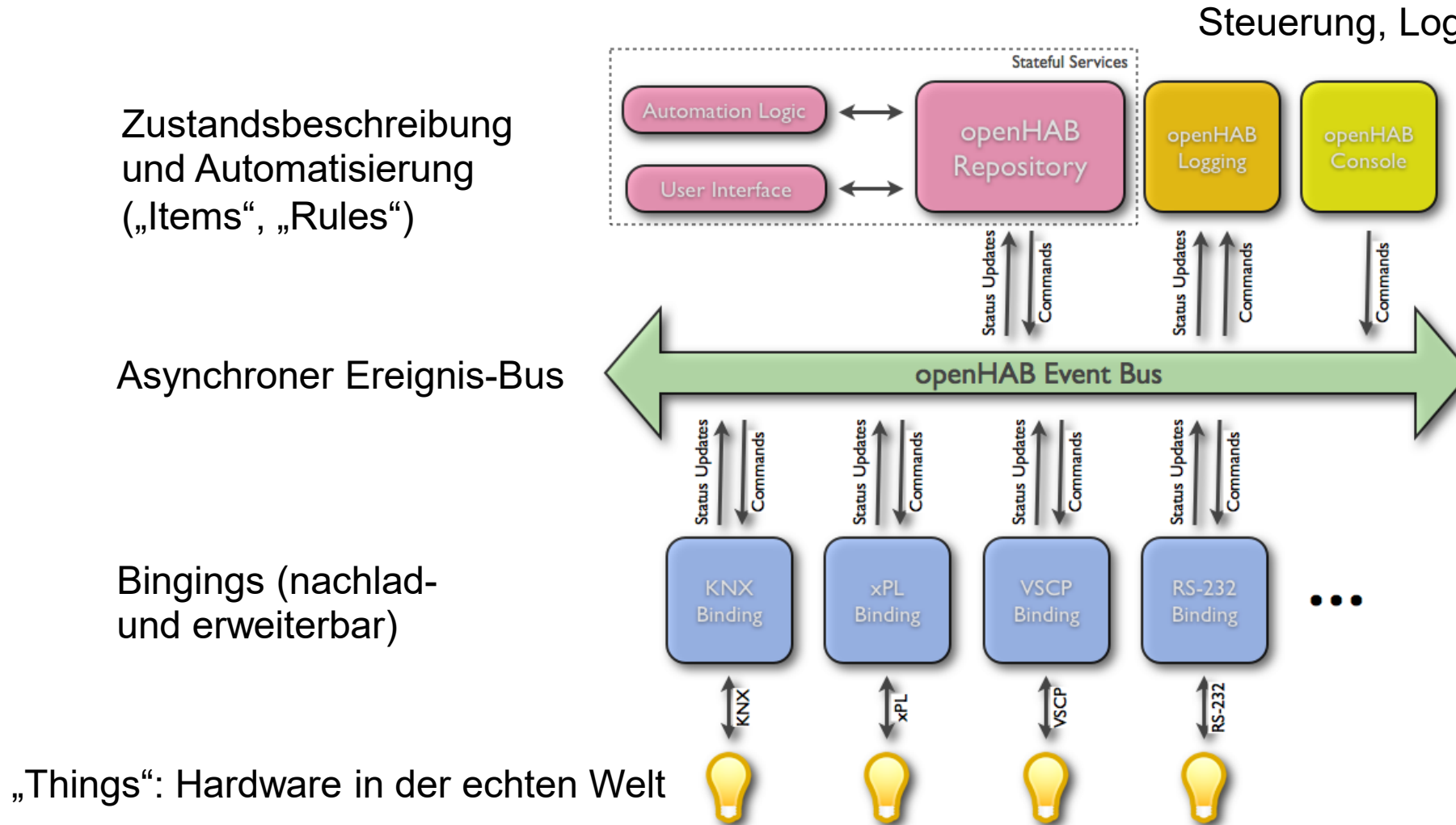
The script is composed of several blocks:

- Logic** block: `item MyItem`
- Text** block: `get item state` (input: `item MyItem`)
- Text** block: `send command` (input: `value`) to `item MyItem`
- Text** block: `log error` (input: `abc`)
- Text** block: `print` (input: `abc`)

The script also includes a **to control the climate in the room** block with the following logic:

- set** `temperatureItem` to `"Temperature"`
- set** `windowItem` to `"Window"`
- set** `heatingItem` to `"Heating"`
- to** `temperatureItem` **append text** `currentRoom`
- to** `windowItem` **append text** `currentRoom`
- to** `heatingItem` **append text** `currentRoom`
- if** `windowItem` **is** `"Window_FF_Office"`
 - do** `to windowItem` **append text** `"_Window"`
- if** `get item state` `temperatureItem` **is less than** `"19"`
 - do**
 - if** `room has window` with `roomWindow` `currentRoom` **and** `get item state` `windowItem` **is** `"OPEN"`
 - do** `log info` `create text with` `"Window open, not starting heating:"` `currentRoom`
 - else** `log info` `create text with` `"Temperature too low, start heating:"` `currentRoom` **send command** `"ON"` to `heatingItem`
 - else if** `room has window` with `roomWindow` `currentRoom`

OpenHAB



Von Kai Kreuzer openHAB Entwickler - Eigenes [Werk](https://github.com/openhab/openhab/wiki/images/events.png)
Copyrighted free use, <https://commons.wikimedia.org/w/index.php?curid=32157828>

OpenHAB gliedert sich in
Ausführender Server-Prozess: openHAB-runtime
„Benutzerfreundliche“ Konfigurationsoberfläche

Version 1.X und früher ist nur Server, die Konfiguration geht nur über Konfigurationsdateien, die „von Hand“ editiert werden müssen.

Version 2.X bietet eine Web-basierte Konfigurationsoberfläche an; trotzdem müssen Konfigurationsdateien noch editiert werden.

Die Konfiguration einer openHAB Lösung ist noch etwas für Experten, die Bedienung ist dann intuitiv über Web-Oberflächen.

OpenHAB Konfigurationsdateien

z.B. für Linux (Raspberry Pi) in den Verzeichnissen

/etc/openhab2/items

/etc/openhab2/sitemaps

/etc/openhab2/things

/etc/openhab2/rules

Text-Dateien mit den Endungen .items, .sitemap, .things, .rules

Concepts	Meaning
Bindings	are the openHAB component that provides the interface to interact electronically with devices
Things	are the first openHAB (software) generated representation of your devices
Items	are the openHAB (software) generated representation of information about the devices
Channels	are the openHAB (software) connection between “Things” and “Items” (see below)
Rules	that perform automatic actions (in its simplest form: if "this" happens, openHAB will do "that")
Pages	is the openHAB (software) generated user interface (web site) that presents information and allows for interactions

OpenHAB Konfigurationsdateien

Beispiel: Home.items Datei

Alle Objekte werden in den .items Dateien definiert (ähnlich einer Variablen- und Objekt-Deklaration)

```
Group Home          "Our Home"    <house>
Group Library       "Library"     <office>    (Home)
Group LivingRoom    "Living Room"  <sofa>      (Home)

Switch Library_Light "Light"      <light>      (Library, gLight)    {channel=""}
```

```
Number Library_Heating "Heating"  <heating>   (Library, gHeating)  {channel=""}
```

```
Number Library_Humidity "Humidity" <humidity> (Library, gHumidity) {channel=""}
```

```
Number Library_Temperature "Temperature" <temperature> (Library, gTemperature) {channel=""}
```

```
Switch LivingRoom_Light "Light"    <light>    (LivingRoom, gLight)  {channel=""}
```

```
Number LivingRoom_Heating "Heating"  <heating>   (LivingRoom, gHeating) {channel=""}
```

```
Number LivingRoom_Temperature "Temperature" <temperature> (LivingRoom, gTemperature) {channel=""}
```

```
Number LivingRoom_Humidity "Humidity"  <humidity> (LivingRoom, gHumidity) {channel=""}
```



```
Group:Switch:OR(ON, OFF) gLight "Light"    <light>    (Home)
```

```
Group:Number:AVG gHeating "Heating"  <heating>   (Home)
```

```
Group:Number:AVG gHumidity "Humidity"  <humidity>   (Home)
```

```
Group:Number:AVG gTemperature "Temperature" <temperature> (Home)
```

OpenHAB Konfigurationsdateien

Lichtschalter als Item:

```
Switch Library_Light      "Light"      <light>      (Library, gLight)      {channel=""}
```

Objekt ist ein Schalter (Switch) mit dem Namen („Variablennamen“) Library_Light und dem Label „Light“ (im Web-Interface sichtbare Bezeichnung), dem Icon <light>.

Das Objekt gehört zu den Gruppen (Library, gLight).

Mit dem Objekt ist die in {} eingetragene Aktion verknüpft (in diesem Fall nichts).

OpenHAB Konfigurationsdateien

Beispiel: Home.sitemap Datei beschreibt die Struktur.

```
sitemap our_home label="Our Home" {  
    Frame {  
        Group item=Library  
        Group item=LivingRoom  
    }  
    Frame {  
        Text label="Light" icon="light" {  
            Default item=Library_Light label="Library"  
            Default item=LivingRoom_Light label="Living Room"  
        }  
        Text label="Heating" icon="heating" {  
            Default item=Library_Heating label="Library"  
            Default item=LivingRoom_Heating label="Living Room"  
        }  
        Text label="Humidity" icon="humidity" {  
            Default item=Library_Humidity label="Library"  
            Default item=LivingRoom_Humidity label="Living Room"  
        }  
        Text label="Temperature" icon="temperature" {  
            Default item=Library_Temperature label="Library"  
            Default item=LivingRoom_Temperature label="Living Room"  
        }  
    }  
}
```

HomeBuilder

Please select your language
German

Home Setup Name
wohnung_admin

Floors
Floors
Draußen x Erdgeschoss x

Rooms
Draußen
Lagerraum x

Erdgeschoss:
Gang x Schlafzimmer x Wohn-Esszimmer x
Büro x Lagerraum x Badezimmer x
Toilette x

Objects
Lagerraum (Draußen)
Type to search or add object
Gang (Erdgeschoss)
Type to search or add object
Schlafzimmer (Erdgeschoss)
Type to search or add object

ITEMS
SITEMAP

Badezimmer
Schlafzimmer
Gang
Wc
Abstellraum
Büro
Wohn-Esszimmer
Balkon
Lagerraum

DIGITALE WELT
www.digitalewelt.at

HomeBuilder

Please select your language

German

Home Setup Name

wohnung_admin

Floors

Floors

Draußen

Erdgeschoss

Rooms

Draußen

Lagerraum

Erdgeschoss

Gang

Schlafzimmer

Wohn-Esszimmer

Büro

Lagerraum

Badezimmer

Toilette

Objects

Lagerraum (Draußen)

Type to search or add object

Gang (Erdgeschoss)

Type to search or add object

Schlafzimmer (Erdgeschoss)

Type to search or add object

ITEMS

Copy

Group	Home	"wohnung_admin"	<house>	
Group	OU	"Draußen"	<garden>	(Home)
Group	GF	"Erdgeschoss"	<groundfloor>	(Home)
Group	OU_StorageRoom	"Lagerraum"	<suitcase>	(Home, OU)
Group	GF_Corridor	"Gang"	<corridor>	(Home, GF)
Group	GF_Bedroom	"Schlafzimmer"	<bedroom>	(Home, GF)
Group	GF_LivingDining	"Wohn-Esszimmer"	<sofa>	(Home, GF)
Group	GF_Office	"Büro"	<office>	(Home, GF)
Group	GF_StorageRoom	"Lagerraum"	<suitcase>	(Home, GF)
Group	GF_Bathroom	"Badezimmer"	<bath>	(Home, GF)
Group	GF_Toilet	"Toilette"	<toilet>	(Home, GF)

SITEMAP

DIGITALE WELT
www.digitalewelt.at

OpenHAB Konfigurationsdateien

Beispiel: myHouse.items

Hier wird ein MQTT Binding eingesetzt, um mit das EPS8266 Board mit seinen Sensoren einzubinden.

Group	My	"EPS8266"	<office>	
Switch	SchalterRoteLED	"RoteLED"	<light>	(myHouse)
		{ mqtt=">[mosquitto:LEDonBoard:command:ON:true],>[mosquitto:LEDonBoard:command:OFF:false]" }		
Number	My_Humidity	"Humidity [%.2f rel. percent]"	<humidity>	(myHouse)
		{mqtt="<[mosquitto:Sensors/Humidity:state:default]"}		
Number	My_Temperature	"Temperature [%.1f °C]"	<temperature>	(myHouse)
		{mqtt="<[mosquitto:Sensors/Temperature:state:default]"}		
Number	My_Pressure	"Luftdruck [%.0f Pa]"	<pressure>	(myHouse)
		{mqtt="<[mosquitto:Sensors/Pressure:state:default]"}		

OpenHAB Rules

Dateien `RegelName_oder_so.rules` in `/etc/openhab2/rules`

Deklaration von Variablen (gelten innerhalb einer Regel-Datei):

```
var counter = 0           // a variable with an initial value. Note that the variable type is automatically inferred
val msg = "This is a message" // a read-only value, again the type is automatically inferred
```

Regeln:

```
rule "<RULE_NAME>"
when
    <TRIGGER_CONDITION> [or <TRIGGER_CONDITION2> [or ...]]
then
    <SCRIPT_BLOCK>
end
```

OpenHAB Rules

Ereignisse können Regeln auslösen (triggern):

Item <item> received command [<command>]

Item <item> received update [<state>]

Item <item> changed [from <state>] [to <state>]

Von Timer ausgelöste Regeln:

Time is midnight

Time is noon

Time cron "<cron expression>"

... und weitere Möglichkeiten

OpenHAB Rules

Beispiel Dimmer:

```
rule "Dimmen Light"
when
    ITEM DimmndLight received command
then
    var Number percent = 0
    if(DimmndLight.state instanceof DecimalType) percent = DimmndLight.state as DecimalType

    if(receivedCommand == INCREASE) percent = percent + 5
    if(receivedCommand == DECREASE) percent = percent - 5

    if(percent < 0) percent = 0
    if(percent > 100) percent = 100

    postUpdate(DimmndLight, percent);
End
```

OpenHAB Rules

Beispiel: „Channel“ des „Astro“ Binding löst eine Regeln aus:

```
rule "Start wake up light on sunrise"  
when  
    Channel "astro:sun:home:rise#event" triggered START  
then  
    ...  
End
```

Online Documents

MQTT

<https://www.heise.de/developer/artikel/MQTT-Protokoll-fuer-das-Internet-der-Dinge-2168152.html>

Node-Red

<https://nodered.org/>

JSON

<https://www.json.org/>

<https://en.wikipedia.org/wiki/JSON>

https://de.wikipedia.org/wiki/JavaScript_Object_Notation

openHAB

<https://docs.openhab.org/>

<https://docs.openhab.org/concepts/index.html>

<https://docs.openhab.org/tutorials/beginner/index.html>

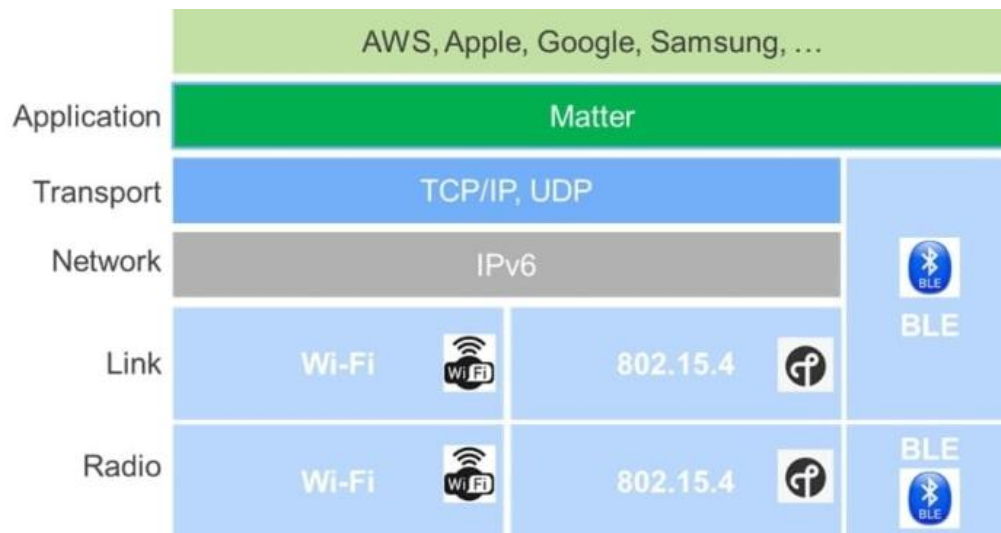
Literaturempfehlung für den tieferen Einstieg: Walter Trojan, Das MQTT-Praxisbuch – mit ESP8266 und Node-RED (elektor Verlag, 2017, 1. Auflage)

Was ist Matter?

Matter ist ein offener, IP-basierter Kommunikationsstandard für Smart-Home-Geräte, der von der Connectivity Standards Alliance (CSA) spezifiziert wird.

Ziel ist die herstellerübergreifende Interoperabilität:

Ein zertifiziertes Matter-Gerät soll mit beliebigen kompatiblen Plattformen – **etwa Apple Home, Google Home oder Amazon Alexa** – zusammenarbeiten, ohne proprietäre Bridges oder Hersteller-Apps zu erfordern.



Info Kernaussage: Matter ist die gemeinsame Anwendungssprache im Smart Home – kein Funkmedium, sondern ein Protokoll auf der Anwendungsschicht, das auf bestehenden IP-Netzen (Wi-Fi, Thread, Ethernet) aufbaut.

- Offener Standard: Spezifikation frei zugänglich, Implementierungen unter Lizenz
- IP-basiert: Nutzung von IPv6 als gemeinsame Netzwerkbasis
- Lokale Kommunikation: Steuerung im LAN ohne Cloud-Zwang möglich
- Einfache Inbetriebnahme (Commissioning) via Bluetooth Low Energy

Grundidee und Motivation

Ausgangslage: Fragmentierung

- Smart-Home-Ökosysteme waren lange herstellerzentrisch und inkompatibel
- Jeder Hersteller pflegte eigene Apps, Cloud-Dienste und Protokolle
- Insel-Lösungen: Geräte verschiedener Marken funktionierten nicht zusammen
- Nutzer mussten zwischen Plattformen wechseln
- Hoher Integrationsaufwand für Entwickler und Endanwender

Zielbild: Einheitliche Interoperabilität

- Matter soll Fragmentierung strukturell reduzieren
- Ein Gerät kann in mehreren kompatiblen Ökosystemen gleichzeitig genutzt werden (Multi-Admin)
- Hersteller entwickeln einmalig gegen die Matter-Spezifikation
- Geringere Zertifizierungskosten und breiterer Marktzugang
- Nutzer erhalten mehr Wahlfreiheit bei Plattform und Gerät

Technische Einordnung von Matter

Matter ist als Anwendungsschichtstandard im OSI-Modell einzuordnen. Es definiert Gerätemodelle, Datentypen und Interaktionsprotokolle – nicht die darunter liegende Funktechnik.

Schicht

Anwendungsschicht (Layer 7 im OSI-Modell)

Definiert Gerätetypen, Cluster, Attribute und Befehle

Netzwerkbasis

Internet Protocol **IPv6**

Unterstützt Wi-Fi, Ethernet und Thread als Transportmedien

Commissioning

Inbetriebnahme via Bluetooth Low Energy (BLE)

QR-Code oder NFC zur sicheren Gerätepaarung

Sicherheit

TLS / CASE-Protokoll für Ende-zu-Ende-Verschlüsselung

Passcode-basierte Authentifizierung bei der Inbetriebnahme



Merksatz: Matter sitzt über dem Transport- und Netzwerkstack, nicht an Stelle der Funktechnik.

Thread und Wi-Fi sind die Träger – Matter ist die Sprache.

Commissioning – Wie wird ein Gerät eingebunden?

Commissioning bezeichnet den Prozess, bei dem ein neues Matter-Gerät (Commissionee) einem bestehenden Netzwerk beigegeben und mit Fabric-Credentials ausgestattet wird. Der Commissioner (z.B. eine App oder ein Hub) steuert den gesamten Ablauf.

Step 1 – Gerät-Discovery

BLE-Advertisement oder DNS-SD: Das neue Gerät kündigt sich an. Der Commissioner erkennt es anhand des Discriminators im Onboarding-Payload.

Step 3 – Geräteinformationen lesen

Commissioner liest Descriptor Cluster, Basic Information Cluster (Vendor ID, Product ID, Seriennummer) und Gerätetyp.

Step 5 – Netzwerk-Credentials übertragen

Wi-Fi-Zugangsdaten oder Thread-Credentials werden sicher an das Gerät übermittelt. Das Gerät verbindet sich mit dem operativen IP-Netz.

Step 2 – PASE (Passcode Authenticated Session Establishment)

Sichere Verbindung via BLE mit dem Passcode aus QR-Code oder Manual Pairing Code. Ein Fail-Safe wird gesetzt, um bei Abbruch zurückzurollen.

Step 4 – Attestation (Geräte Zertifizierung)

Prüfung des Device Attestation Certificate (DAC) gegen die CSA-Zertifikatskette. Sicherstellung, dass das Gerät ein echtes, zertifiziertes Matter-Produkt ist.

Step 6 – CASE & Fabric-Aufnahme

Certificate Authenticated Session Establishment (CASE): Das Gerät erhält seinen Node-ID und Fabric-Schlüssel. Commissioning abgeschlossen – Gerät ist operativ im Fabric.

 **Onboarding-Payload:** Der QR-Code oder Manual Pairing Code enthält Vendor ID, Product ID, 12-Bit-Discriminator und 27-Bit-Passcode – alle nötigen Daten für den sicheren Einstieg.

Was ist ein Fabric?

Ein Fabric ist das logische Vertrauensnetzwerk in Matter – eine gemeinsame Sicherheitsdomäne, in der alle Teilnehmer (Geräte, Controller, Apps) mit denselben kryptografischen Schlüsseln und Zertifikaten arbeiten.

Technische Definition

- Jedes Fabric hat eine eindeutige Fabric-ID und einen gemeinsamen Root-of-Trust (Stammzertifikat)
- Jedes Gerät im Fabric erhält eine eindeutige Node-ID und ein Operational Certificate (NOC)
- Kommunikation innerhalb des Fabrics läuft über CASE (Certificate Authenticated Session Establishment)
- Ein Gerät kann gleichzeitig Mitglied mehrerer Fabrics sein (Multi-Fabric = Grundlage für Multi-Admin)

Fabric und Multi-Admin

- Apple Home, Google Home und Amazon Alexa betreiben jeweils ihr eigenes Fabric
- Ein Matter-Gerät kann in alle drei Fabrics gleichzeitig eingebunden sein
- Jede Plattform steuert das Gerät unabhängig über ihr eigenes Fabric
- Kein Datenaustausch zwischen den Fabrics – jedes Fabric ist isoliert und sicher
- Multi-Admin = ein Gerät, mehrere Fabrics, mehrere Plattformen gleichzeitig

 **Fabric ≠ Netzwerk:** Ein Fabric ist keine physische Netzwerktopologie, sondern eine kryptografische Vertrauensdomäne. Mehrere Fabrics können dasselbe physische IP-Netz nutzen, ohne sich gegenseitig zu sehen.

Architektur in vereinfachter Sicht

Die Matter-Architektur gliedert sich in standardisierte Knoten, IP-basierte Kommunikationspfade und plattformübergreifende Steuerinstanzen.

Geräte und Kommunikation

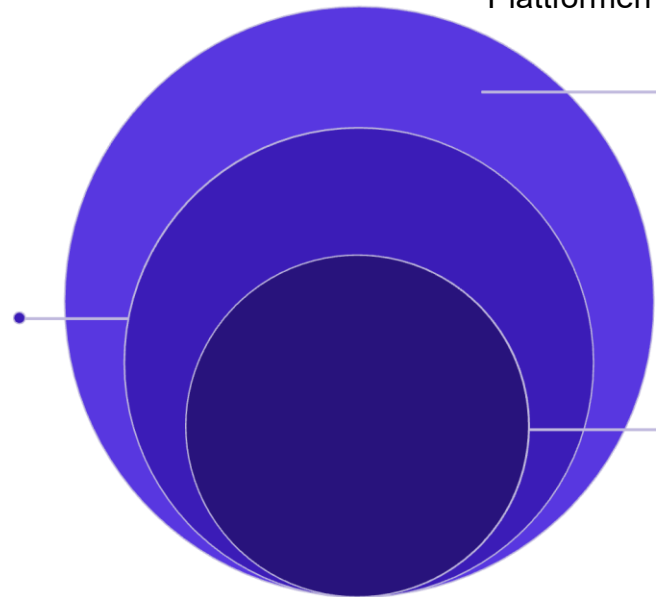
- **Matter-Knoten** sind Endgeräte mit standardisierten Geräteklassen (z.B. Licht, Schalter, Sensor)
- Jeder Knoten exponiert **Cluster** – definierte Gruppen von Attributen und Befehlen
- Kommunikation läuft vollständig IP-basiert im lokalen Netz
- Steuerung durch Controller, Apps, Hubs oder Plattform-Bridges

Architekturkonzepte

- **Lokale Steuerung:** Befehle werden ohne Cloud-Umweg verarbeitet
- **Standardisierte Gerätemodelle:** Einheitliche Semantik für alle Plattformen
- **Zertifizierte Interoperabilität:** CSA-Zertifizierung gewährleistet Konformität
- **Multi-Admin:** Gleichzeitige Nutzung eines Geräts durch mehrere Plattformen möglich

Netzwerkschicht

IPv6-basierte End-to-End-Kommunikation



Transportschicht

Wi-Fi, Thread und Ethernetverbindungen

Matter-Anwendung

Gerätemodelle, Cluster und Befehle

Matter vs. Zigbee

Matter

- IP-basierter Anwendungsschichtstandard
- Fokus auf plattformübergreifende Interoperabilität
- Läuft auf Thread, Wi-Fi oder Ethernet
- Neuere Ökosystem, wachsende Gerätebasis
- Multi-Admin nativ im Protokoll verankert
- Standardisiertes Commissioning via BLE

Zigbee

- Eigenständiger Smart-Home-Funkstandard (IEEE 802.15.4)
- Nicht IP-nativ; eigenes Mesh-Netzwerkmodell
- Sehr breite installierte Basis, viele etablierte Geräteklassen
- Interoperabilität oft über herstellerspezifische Bridges
- Gut etabliert im professionellen und Consumer-Segment
- Robuste Mesh-Topologie mit langer Praxisbewährung

Fazit: Matter zielt strukturell auf einheitliche Interoperabilität ohne Bridge-Abhängigkeit. Zigbee ist in vielen bestehenden Installationen weiterhin dominant und funktioniert zuverlässig, erfordert jedoch häufig herstellerspezifische Integration.

Matter vs. Thread vs. Wi-Fi

Ein häufiges Missverständnis: Matter, Thread und Wi-Fi operieren auf verschiedenen Schichten des Netzwerkstacks und erfüllen unterschiedliche Funktionen.

Matter

Schicht: Anwendungsprotokoll

- Beschreibt Gerätefunktionen, Interaktionen und Datenmodelle
- Herstellerunabhängige Semantik für Befehle und Attribute
- Läuft auf Thread oder Wi-Fi als Träger

Thread

Schicht: Netzwerk- und Mesh-Technologie

- IPv6-basiertes Low-Power-Mesh-Protokoll (IEEE 802.15.4)
- Energieeffizient, selbstheilend, kein Single Point of Failure
- Bevorzugtes Trägerprotokoll für batteriebetriebene Matter-Geräte

Wi-Fi

Schicht: Drahtloses IP-Netzwerk

- Hohe Bandbreite, weit verbreitet
- Geeignet für leistungsfähige Matter-Geräte (Kameras, Displays)
- Höherer Energieverbrauch als Thread

❏ **Kerngedanke:** Matter ersetzt Thread und Wi-Fi nicht – es nutzt sie als Transportmedien. Die Wahl des Funkmediums beeinflusst Energieeffizienz, Reichweite und Latenz; Matter selbst bleibt davon unabhängig.



Vorteile von Matter

Interoperabilität

- Einheitlicher Gerätezugang über Plattformgrenzen hinweg
- Weniger Ökosystem-Silos und proprietäre Abhängigkeiten
- Multi-Admin: ein Gerät, mehrere Plattformen gleichzeitig

Lokale Kommunikation

- Steuerung im LAN ohne Cloud-Abhängigkeit
- Potenziell geringere Latenz bei direkter IP-Kommunikation
- Höhere Robustheit: Funktioniert auch bei Internet-Ausfall

Herstellerperspektive

- Einmalige Implementierung für mehrere Plattformen
- Vereinfachte Zertifizierung über die CSA
- Breiterer Marktzugang mit reduziertem Entwicklungsaufwand

Interoperabilität

Einheitlicher Gerätezugang über Plattformgrenzen hinweg

Nachteile und Grenzen von Matter

Abdeckung und Reifegrad

- Noch nicht alle Gerätetypen vollständig spezifiziert (z.B. Kameras, komplexe Aktoren in alten Versionen)
- Reale Unterstützung hängt stark von Matter-Version und Hersteller-Implementierung ab
- Versionsdrift: Nicht jede Plattform unterstützt den aktuellen Funktionsumfang

Hardwareanforderungen

- Höhere Anforderungen an Speicher und Rechenleistung gegenüber einfachen proprietären Protokollen
- Unterstützte Funktechnik (Thread-Radio oder Wi-Fi) muss im Gerät vorhanden sein
- Ältere oder günstige Hardware ggf. nicht nachrüstbar

Netzwerk und Infrastruktur

- Matter beseitigt keine physikalischen Funkprobleme
- Schlechte Wi-Fi-Planung oder schwaches Thread-Mesh bleiben ungelöst
- Thread benötigt Border Router als Gateway ins IP-Netz
- Komplexere Netzwerkkonfiguration bei Mischinstallationen



Merksatz: Matter verbessert Interoperabilität auf Anwendungsebene, löst aber keine Probleme auf Netzwerk- oder Funkebene. Funkqualität, Mesh-Planung und Hardware-Reife bleiben eigenständige Herausforderungen.

Praktische Einsatzszenarien

1

Neuinstallationen

Besonders geeignet: Neue Smart-Home-Setups ohne Legacy-Geräte

- Volle Ausnutzung der Matter-Interoperabilität von Beginn an
- Kombination aus Thread-Mesh und Wi-Fi je nach Gerätekategorie optimal planbar

2

Multi-Plattform-Haushalte

Sinnvoll: Haushalte mit gemischten Plattformpräferenzen

- Apple Home, Google Home und Alexa parallel nutzbar
- Multi-Admin ermöglicht plattformübergreifende Steuerung ohne doppelte Hardware

3

Legacy-Bestände

Eingeschränkt: Installationen mit vielen Zigbee- oder proprietären Geräten

- Direkte Matter-Nutzung nicht möglich ohne Hardware-Austausch
- Bridges (z.B. Philips Hue Bridge mit Matter-Support) als Übergangslösung

4

Hybridstrategie

Praxisstrategie: Schrittweise Migration und Parallelbetrieb

- Matter-fähige Geräte beim Neukauf bevorzugen
- Bridges für Bestandsgeräte einsetzen, solange wirtschaftlich sinnvoll

i Fazit: Matter ist in der Praxis häufig Ergänzung und Migrationspfad – kein sofortiger Komplettersatz bestehender Installationen.

Zusammenfassung und Bewertung

Technische Kernpunkte

- Matter ist ein offener, IP-basierter Anwendungsschichtstandard für das Smart Home
- **Wichtige Abgrenzung:** Matter ist nicht Thread und nicht Wi-Fi – es nutzt diese als Transportmedien
- Standardisierte Gerätemodelle (Cluster) ermöglichen semantisch einheitliche Kommunikation
- Commissioning über BLE, Betrieb über IPv6

Stärken

- Plattformübergreifende Integration (Multi-Admin)
- Lokale Kommunikation ohne Cloud-Zwang
- Geringere Herstellerbindung, offene Spezifikation

Schwächen

- Begrenzte Geräteabdeckung in frühen Versionen
- Höhere Hardwareanforderungen
- Keine Lösung für Funk- oder Netzwerkprobleme

Gesamtbewertung

Strategische Relevanz: Sehr hoch – breite Industrieunterstützung durch Apple, Google, Amazon, Samsung

Praktische Verbreitung: Stark wachsend – Gerätebasis und Plattformunterstützung expandieren kontinuierlich

Reale Architekturen: Häufig hybride Setups mit Matter-Geräten, Bridges für Legacy-Hardware und gestufter Migration

 **Einordnung:** Matter stellt die bedeutendste Standardisierungsinitiative im Consumer-IoT-Bereich der letzten Jahre dar und wird mittelfristig zur dominierenden Interoperabilitätsgrundlage im Smart Home.
→ Evtl. Sehr auf wenige BigPlayer fokussiert

OPC UA – Zielsetzung

Grundidee

- Herstellerunabhängige M2M-Kommunikationsarchitektur
- Zentrale Abstraktions- und Integrationsschicht im industriellen Kontext
- Ziel: nicht nur Datentransport, sondern **strukturierte, standardisierte und sichere Interoperabilität**

Abgrenzung

- Unterscheidet sich deutlich von transport- oder message-orientierten Protokollen
- Informationsmodell ist integraler Bestandteil des Standards
- Sicherheit ist kein Add-on, sondern von Grund auf integriert

 OPC UA = Datenmodell + Dienste + Sicherheit + Transport in einem Standard.

OPC UA als Abstraktionsarchitektur

Serverseite

- OPC-UA-Server stellt Daten vereinheitlicht bereit
- Geräte und Steuerungen kommunizieren nicht mehr über viele Einzeltreiber
- Vermittlung zwischen heterogenen Datenquellen

Clientseite

- OPC-UA-Client greift auf standardisierte Dienste zu
- Anwendungen wie SCADA, HMI oder Analytiksysteme werden entkoppelt
- Geringere Integrationskomplexität bei heterogenen Geräteumgebungen

Bausteine von OPC UA

Basisdienste

- Standardisierte Servicefunktionen
- Lesen, Schreiben, Subscriptions
- Discovery und Browsing

Informationsmodelle

- Strukturierte Repräsentation von Objekten
- Beziehungen zwischen Knoten
- Typsystem und Vererbung

Transportprofile

- UA TCP
- HTTP / HTTPS
- UA Binary, UA XML

Erweiterbarkeit

- Anbieter- und domänenspezifische Modelle möglich
- Companion Specifications (z. B. für Robotik, Antriebstechnik)

OPC UA und Sicherheit

Sicherheitsarchitektur

- Sicherheit ist integraler Bestandteil des Standards
- Trennung zwischen Kommunikationsschicht und Anwendungsschicht
- Sicherer Kanal als Grundlage der Kommunikation

Kommunikationsschicht

- Vertraulichkeit (Verschlüsselung)
- Integrität (Signatur)
- Anwendungsauthentifizierung

Anwendungsschicht

- Nutzerauthentifizierung
- Autorisierung (rollenbasiert)
- Auditing und Protokollierung

Vergleich: HTTP/REST, CoAP, MQTT und OPC UA

Kriterium	HTTP/REST	CoAP	MQTT	OPC UA
Kommunikationsmuster	Request/Response	Request/Response	Publish/Subscribe	Client/Server + Pub/Sub
Overhead	Hoch	Gering	Gering	Mittel bis hoch
Interoperabilität	Hoch (Web-Standard)	Mittel	Mittel	Sehr hoch (industriell)
Semantik	Gering	Gering	Gering	Sehr hoch (Informationsmodell)
Sicherheit	TLS/HTTPS	DTLS	TLS	Integriert im Standard
Architekturrolle	API-zentriert	Leichtgewichtiges REST	Ereignisgetrieben	Dienst- und modellgetrieben

Systematische Einordnung – Schlussbild

01

Schichtenmodelle

Strukturieren IoT-Systeme von der Geräteebe-
ne bis zur Business- und Dienstebene

02

Web Calls und Datenformate

Zentrale Bausteine API-orientierter Integration;
Formatwahl beeinflusst Overhead und
Interoperabilität

03

CoAP, MQTT, OPC UA

Adressieren unterschiedliche
Architekturprobleme – keine universelle Lösung,
sondern anforderungsgetriebene Wahl

 Leitfrage für Klausur und Diskussion: Welches Protokoll passt zu welchem Kommunikationsmuster und welcher Systemarchitektur?